

APB-CONTROLLED I²C

MASTER AND SLAVE SUBSYSTEM

RTL Design · Integration · Verification · Results

PREPARED BY

Yousef Hatem Abdeljalil
Omar Ahmed Gamal Abdo
Ahmed Wael Sadek EL Sawy
Zyad Hany Mohamed
Youssef Mohamed Zain

Table of Contents

Table of Contents.....	i
Table of Figures.....	iii
Table of Tables.....	iv
Team Responsibilities	v
Terms and Abbreviations	v
1 Introduction	1
1.1 Objective	1
1.2 Design philosophy	1
1.3 Scope.....	2
2 I ² C Protocol Background	5
3 System Architecture.....	6
3.1 Module hierarchy.....	6
3.2 Design rationale	7
4 APB Register File	8
4.1 Transfer model.....	8
4.2 Error response policy	8
4.3 Register map	8
4.4 Register bit definitions	9
4.4.1 CTRL — 0x00	9
4.4.2 STATUS — 0x04.....	9
4.4.3 Remaining registers.....	9
4.5 Software usage sequence	10
5 Clocking, Reset and Timing	11
5.1 Clock domain.....	11
5.2 Clock-enable generation	11
5.3 Frequency derivation	11
5.4 Standard-mode timing compliance.....	12
5.5 Reset strategy	12
5.6 Slave synchroniser latency	13
6 I ² C Master Design.....	14
6.1 Responsibilities	14
6.2 FSM state diagram	14
6.3 State table	15
6.4 Acknowledge handling	15
6.5 Status signal semantics	16
7 I ² C Slave Design	17

7.1 Responsibilities	17
7.2 Bus observation and condition detection	17
7.3 Priority structure	17
7.4 FSM state diagram	18
7.5 State table	20
7.6 The ack_seen mechanism	21
7.7 Read-path bit presentation	21
8 Shared Bus and Open-Drain Behaviour.....	22
8.1 Electrical model.....	22
8.2 Implementation	22
9 Top-Level Integration — Top_Module.....	23
9.1 External interface.....	23
9.2 Internal interconnect	23
9.3 Instantiation and bus formation	24
9.4 How a transaction flows through the top level	25
10 Verification and Simulation Results	26
10.1 Strategy	26
10.2 APB register-file verification	27
10.3 I ² C master waveforms	32
10.4 I ² C slave waveforms	36
11 Coverage Analysis	41
11.1 Functional coverage	41
11.2 Code coverage.....	41
11.3 Analysis of the toggle gap	41
12 Future Enhancements	43
13 Conclusion	44
Appendix A — Simulation Transcript Summaries	45
A.1 I ² C master testbench.....	45
A.2 I ² C slave testbench.....	45
Appendix B — FSM Encodings	46

Table of Figures

— System-level view of the APB register file, the I ² C master and slave, and the shared open-drain bus.....	6
— I ² C master FSM. Transitions are gated by i2c_tick; the highlighted path is the NACK abort route.	14
— I ² C slave FSM, high-level view. The dashed paths are the global START/STOP overrides.	18
— Panel 1 — top-level priority: reset, START, STOP, then the state case.	18
— Panel 2 — RECV_ADDR bit reception and the ADDR_ACK decision tree.	19
— Panel 3 — WRITE_DATA reception and the DATA_ACK cell.	19
— Panel 4 — READ_DATA transmission, READ_ACK sampling and WAIT_STOP.	20
— APB write transaction: SETUP followed by ACCESS correctly writes the register.	28
— APB read transaction: the read multiplexer drives the correct register value onto PRDATA.	29
— START command pulse: writing CTRL[0] = 1 generates exactly one PCLK cycle of master_start. ...	30
— Status and data feedback captured from the I ² C master into the register block.	30
— An invalid APB address asserts PSLVERR during the ACCESS phase.....	31
— Reset: APB and I ² C-facing signals return to the idle/default condition.	32
— START condition generated by the master.....	33
— STOP condition generated by the master.	33
— Test 1 — WRITE: correct address and data transfer.	33
— Test 2 — READ: correct address and readback.	34
— Test 3 — WRITE: MSB-first bit ordering confirmed.....	34
— Test 4 — WRITE to a wrong address: the expected NACK and clean abort.	35
— Test 5 — multiple consecutive WRITE transactions.	35
— Test 6 — multiple READ transactions returning a stable value.....	36
— Slave testbench: START condition detected from the idle state.....	36
— Slave testbench: STOP condition returns the FSM to IDLE.	36
— Test 1 — WRITE: address matched, byte received and acknowledged.	37
— Test 2 — READ: the stored byte is transmitted MSB-first.....	37
— Test 3 — WRITE with a new data pattern.	38
— Test 4 — wrong address: the slave ignores the transaction until STOP.....	38
— Test 6 — multiple READ transactions.....	39
— Test 7 — second READ confirming the retained data register.....	39
— Test 8 — STOP issued mid-transaction: the slave releases SDA and returns to IDLE.	40

Table of Tables

— Work split across the team.	v
— Terms and abbreviations used throughout this report.	v
— Frozen scope. Items in the right-hand column are future enhancements only.	2
— Mapping of I ² C protocol elements onto the implementation.	5
— Module hierarchy under Top_Module.	6
— APB register map.	8
— CTRL register. W1P = write-1-to-pulse.	9
— STATUS register.	9
— SLAVE_ADDR, TX_DATA, RX_DATA, CLK_DIV, INT_STATUS and VERSION.	9
— Timing derivation for the default configuration.	11
— Standard-mode timing compliance. Values are nominal simulation figures.	12
— Reset behaviour per module, as currently implemented.	12
— I ² C master FSM state table.	15
— Acknowledge cell behaviour.	16
— Slave state-update priority.	17
— I ² C slave FSM state table. Encodings match the numeric states printed in the simulation transcript.	20
— Wired-AND truth table for SDA.	22
— Top_Module port list.	23
— Internal nets inside Top_Module.	24
— Transaction flow through Top_Module.	25
— Verification summary.	26
— Functional coverage summary for apb_i2c_regs.	41
— Code coverage for apb_i2c_regs.	41
— Toggle coverage exclusions.	42
— Future enhancements, ordered by value per unit of effort.	43
— State encodings, matching the numeric states printed in the transcripts.	46

Team Responsibilities

The work was partitioned by module so that the three design blocks could be developed and verified in parallel, then merged at the top level.

Member	Primary responsibility	Secondary contribution
Yousef Hatem Abdeljalil	I ² C SlaveRTL and master FSM	Slave testbench
Omar Ahmed Gamal Abdo	I ² C Master RTL and slave FSM	Master testbench
Ahmed Wael Sadek EL Sawy	APB testbench, coverage closure	Documentation and results review
Zyad Hany	Top module RTL	Testbench & Simulation transcripts and waveforms of top module
Youssef Mohamed Zain	APB Register File	APB Register File testbench

Table 1 — Work split across the team.

Terms and Abbreviations

Term	Meaning
APB	Advanced Peripheral Bus (AMBA APB4), the processor-side control interface.
I ² C	Inter-Integrated Circuit two-wire serial bus (SDA and SCL).
SDA / SCL	Serial data line and serial clock line of the I ² C bus.
START / STOP	Bus conditions marking the beginning and end of an I ² C transaction.
ACK / NACK	Acknowledge (SDA pulled LOW) and not-acknowledge (SDA released).
FSM	Finite state machine.
RTL	Register-transfer level description of the hardware.
W1P	Write-1-to-pulse register field.
DUT	Design under test.

Table 2 — Terms and abbreviations used throughout this report.

1 Introduction

This project implements a complete, synthesisable I²C communication subsystem controlled through an AMBA APB4 register interface. The subsystem contains four cooperating parts: an APB register file that exposes control and status to software, an I²C master that executes bus transactions autonomously, an I²C slave that responds to them, and a shared open-drain bus model that joins the two on the same SDA and SCL lines.

Because no physical hardware was available, the companion slave is not an optional extra; it is what makes the design verifiable. Every transaction the master issues is answered by a real protocol participant rather than by a behavioural stub, so a write can be confirmed by inspecting the slave data register and a read can be confirmed by comparing the byte the master captured against the byte the slave transmitted.

Verification is performed at module level and at the APB register interface using self-checking SystemVerilog testbenches with automated pass/fail reporting. All results quoted in this report are taken from recorded Questa simulation runs.

1.1 Objective

The objective is a small, clean, professional-grade RTL IP block in which an APB4 interface acts as the programming and control path to an I²C bus master. Once launched, the master must carry out a complete I²C transaction without further intervention, reporting progress and completion back through status registers.

1.2 Design philosophy

The project follows five ordered priorities. Where any two conflict, the lower-ranked one yields.

1. Correctness — a transaction that completes must be protocol-legal and must produce the expected data.
2. Clean architecture — clear module boundaries, one responsibility per module, no cross-layer control.
3. Modular RTL — each block compiles, elaborates and simulates standalone.
4. Strong verification — self-checking testbenches with automated pass/fail reporting, not visual waveform inspection.
5. Synthesisability — no simulation-only constructs inside the design hierarchy.

Feature count is explicitly subordinate to all five. A narrower design that is fully verified is a better outcome than a broader design that is only partly exercised.

1.3 Scope

The feature set was frozen at the end of the first planning session and was not revisited, so the boundary between what was built and what was deliberately deferred is unambiguous.

Domain	In scope	Out of scope
APB	APB4-style interface; synchronous operation; 12-bit PADDR; 32-bit PWDATA and PRDATA; PSEL, PENABLE, PWRITE, PREADY, PSLVERR, PCLK, PRESETn.	Wait-stated or pipelined transfers; burst access; DMA; interrupt controller.
I ² C protocol	Standard mode, 100 kHz SCL; 7-bit addressing; 8-bit data; MSB-first; single master; single slave; write and read; ACK, NACK, START, STOP; open-drain SDA and SCL.	Multi-master arbitration; clock stretching; 10-bit addressing; repeated START; multi-byte bursts; Fast mode and above; SMBus; I3C.
Data path	One data byte per transaction.	FIFOs; multi-slave enumeration; scatter-gather.
Verification	Self-checking testbenches; directed tests; bus monitor; scoreboard; functional and code coverage.	Full UVM environment; formal property verification; gate-level simulation.
Implementation	Lint; elaboration; synthesis; utilisation and timing reports.	Bitstream generation; device programming; board validation.

Table 3 — Frozen scope. Items in the right-hand column are future enhancements only.

1.4 Module Interfaces

This section is the authoritative port list. Every signal in the design appears exactly once, with its direction, width and meaning. Interface consistency across files is a stated requirement of the specification; Section 12 records where that consistency is currently broken and what must change.

Top_Module

Signal	Dir	Width	Description
PCLK	in	1	System and APB clock. 50 MHz assumed. Every register in the design is clocked by this signal.
PRESETn	in	1	Active-low reset, APB convention. Inverted internally to produce the active-high rst used by the I ² C blocks.
PADDR	in	12	APB byte address. Only offsets 0x00–0x1C are decoded.
PSEL	in	1	APB select. Asserted for both SETUP and ACCESS phases.
PENABLE	in	1	APB enable. Asserted only during ACCESS.
PWRITE	in	1	1 = write transfer, 0 = read transfer.
PWDATA	in	32	APB write data.
PRDATA	out	32	APB read data. Zero outside an active read.
PREADY	out	1	Transfer-complete. Tied HIGH — every register access completes in the minimum two cycles.
PSLVERR	out	1	Error response. Asserted during ACCESS to an undecoded address.
SDA	inout	1	I ² C serial data. Open-drain; driven LOW or released to Z.

Signal	Dir	Width	Description
SCL	inout	1	I ² C serial clock. Open-drain; driven by the master only in this single-master design.

Table 3.1 — Top_Module port list.

apb_i2c_regs

Parameters: CLK_DIV_DEFAULT = 32'd250 and VERSION_VALUE = 32'h0001_0000.

Signal	Dir	Width	Description
PCLK ... PSLVERR	various	—	APB interface as described in Table 3.1.
master_start	out	1	One-PCLK-cycle command pulse. Asserted when CTRL[0] is written as 1 while the master is not busy.
master_rw	out	1	Transaction direction. 0 = write, 1 = read. Persistent configuration bit, not a pulse.
master_slave_addr	out	7	Target 7-bit I ² C address.
master_tx_data	out	8	Byte to transmit during a write transaction.
master_clk_div	out	32	Divider value passed to the master. Writes of zero are rejected.
master_busy	in	1	Master is executing a transaction.
master_done	in	1	Transaction-complete pulse from the master. Latched into the sticky DONE status bit.
master_ack_error	in	1	Master observed a NACK where an ACK was required.
master_bus_error	in	1	Master entered its ERROR state.
i2c_bus_busy	in	1	Bus-level activity indication. See Section 12 — no source currently drives this signal.
master_rx_data	in	8	Byte captured by the master during a read. Latched on master_done.

Table 3.2 — apb_i2c_regs port list.

i2c_master

Parameters: CLK_FREQ = 50_000_000 and I2C_FREQ = 100_000. These are documentary in the present implementation; the actual tick rate is set at runtime by clk_div.

Signal	Dir	Width	Description
clk	in	1	System clock, 50 MHz.
rst_n	in	1	Synchronous active-low reset. See Section 12 — polarity disagrees with the top level.
start	in	1	Launch pulse. Sampled in IDLE only.
rw	in	1	Direction for the transaction being launched.
slave_addr	in	7	Target address, loaded into the transmit shift register with the R/W bit appended.
tx_data	in	8	Byte to transmit. Loaded on a successful address ACK when rw = 0.
clk_div	in	32	Divider modulus for the tick generator.
rx_data	out	8	Byte received during a read. Updated in MASTER_ACK.
busy	out	1	HIGH from acceptance of start until the FSM reaches DONE.

Signal	Dir	Width	Description
done	out	1	Single-cycle completion pulse.
ack_error	out	1	Sticky within a transaction; set when SDA is sampled HIGH during an ACK cell.
bus_error	out	1	Set in the ERROR state. Currently unreachable — see Section 12.
sda, scl	inout	1 each	Present implementation drives the bus directly through tri-state assigns. Section 12 recommends replacing these with sda_in / scl_in inputs and sda_drive_low / scl_drive_low outputs.

Table 3.3 — *i2c_master* port list, as currently implemented.

i2c_slave

Parameter: SLAVE_ADDRESS = 7'h50.

Signal	Dir	Width	Description
clk	in	1	System clock, 50 MHz. The slave has no I ² C clock generator of its own.
rst	in	1	Synchronous active-high reset.
scl_in	in	1	Observed SCL bus level.
sda_in	in	1	Observed SDA bus level.
sda_drive_low	out	1	1 = pull SDA LOW, 0 = release. The only way this module influences the bus.
rx_data	out	8	Internal data register. Source of read data and destination of write data.

Table 3.4 — *i2c_slave* port list. Note the clean open-drain contract: one input per line, one drive-low output.

Interface contract

The slave interface is the model the master should follow. A module that never drives a line HIGH cannot create bus contention, and a module whose only bus output is a drive-low enable is trivially synthesisable on any technology. The master currently deviates from this pattern; reconciling it is item 1 of Section 12.

2 I²C Protocol Background

I²C is a synchronous serial bus using two lines: SDA for bidirectional data and SCL for the clock. Every device uses an open-drain output with a pull-up resistor, so a released line floats HIGH and any participant may force it LOW. Data moves in 8-bit bytes, and the receiver acknowledges each byte or issues a NACK to terminate a read.

Protocol element	Behaviour in this design
Bus lines	SDA and SCL, shared, with open-drain drive/release behaviour.
Transmission	8-bit bytes, MSB-first; SDA is sampled while SCL is HIGH.
START	SDA falls HIGH-to-LOW while SCL is stable HIGH.
STOP	SDA rises LOW-to-HIGH while SCL is stable HIGH.
Addressing	7-bit slave address followed by the R/W bit.
ACK / NACK	ACK pulls SDA LOW; NACK leaves SDA released.

Table 4 — Mapping of I²C protocol elements onto the implementation.

3 System Architecture

The design is a three-block hierarchy under a single top-level wrapper. Top_Module exposes only the APB pins and the two I²C bus pins; everything else is internal. The APB register file translates software accesses into control values and captures status back. The I²C master converts one control pulse into a full protocol transaction. The I²C slave observes the same two wires and responds to them.

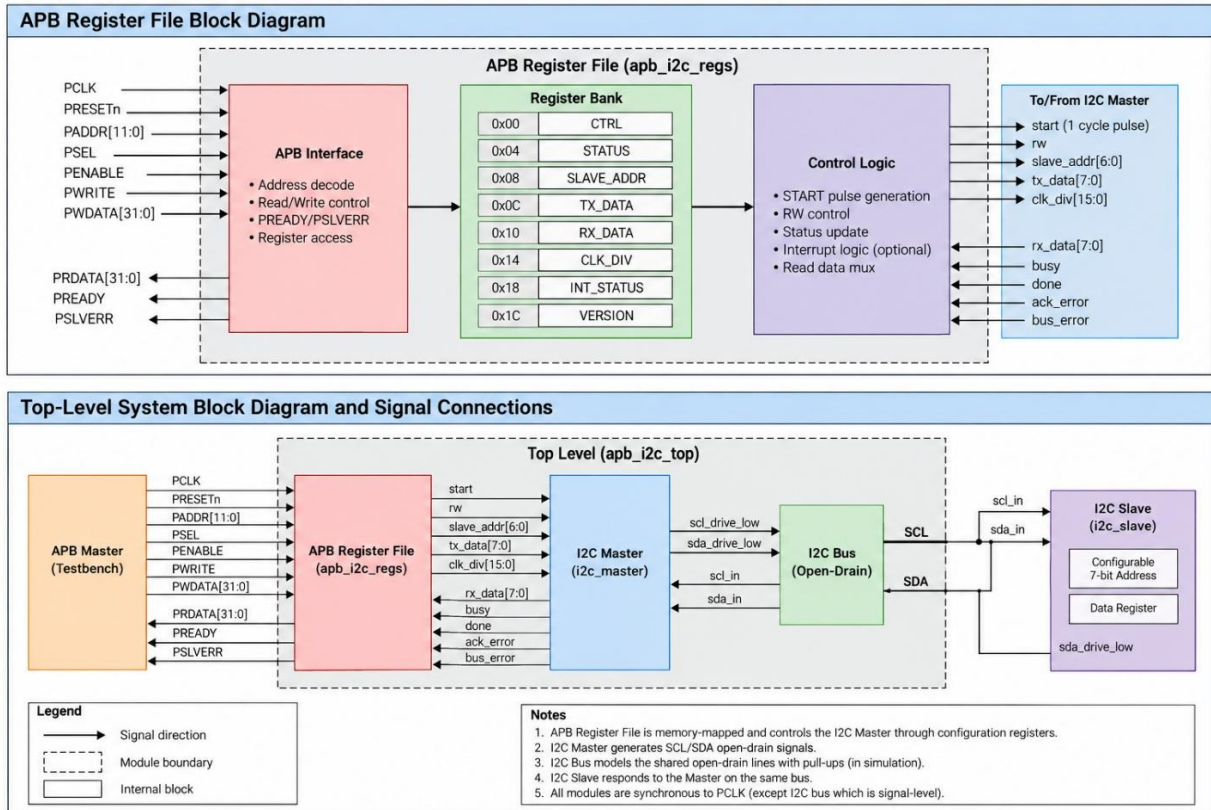


Figure 1 — System-level view of the APB register file, the I²C master and slave, and the shared open-drain bus.

3.1 Module hierarchy

Module	Instance name	Role
Top_Module	—	Top-level wrapper. Exposes APB and I ² C pins, instantiates the three blocks and forms the open-drain bus.
apb_i2c_regs	Bridge	APB4 slave. Address decode, register file, START pulse generation, status capture.
i2c_master	i2c_master	Bus owner. Generates SCL, START and STOP, transmits and receives one byte, reports completion and errors.
i2c_slave	i2c_slave	Pure responder. Parameterised address 0x50; detects START and STOP, matches its address, receives or transmits one byte.

Table 5 — Module hierarchy under Top_Module.

3.2 Design rationale

Keeping the register file and the protocol engine separate is the central architectural decision. The register file knows nothing about I²C timing, and the master knows nothing about APB phases; they communicate through a flat set of control and status wires. This is what allowed three engineers to develop three blocks in parallel, and it is what makes the master reusable behind a different host interface.

4 APB Register File

4.1 Transfer model

The interface implements the standard three-phase APB transfer. There is no pipelining and no wait state: PREADY is tied HIGH, so every access completes in exactly two PCLK cycles.

Writes are qualified by PSEL & PENABLE & PWRITE and by a successful address decode. Reads are combinational — the read multiplexer is a pure function of PADDR, qualified by PSEL & PENABLE & !PWRITE — and PRDATA is driven to zero at all other times. Forcing PRDATA low outside an active read is not required by the protocol, but it makes waveform inspection unambiguous and removes any dependence on stale data.

4.2 Error response policy

A single deterministic policy is used: PSLVERR is asserted during the ACCESS phase of any transfer whose address is not one of the eight decoded offsets. Reads to undecoded addresses additionally return zero, and writes to them have no side effect. Writes to read-only registers — STATUS, RX_DATA, INT_STATUS and VERSION — are decoded as valid addresses, do not assert PSLVERR, and are silently discarded.

This was chosen over returning zero without an error because it gives software an unambiguous signal that an address decode has gone wrong, which is the more useful behaviour during bring-up.

4.3 Register map

Offset	Name	Access	Reset	Description
0x00	CTRL	R/W	0x0000_0000	Transaction control. Write-1-to-START; RW selects direction.
0x04	STATUS	RO	0x0000_0000	Live and sticky status from the I ² C master.
0x08	SLAVE_ADDR	R/W	0x0000_0000	Target 7-bit I ² C address.
0x0C	TX_DATA	R/W	0x0000_0000	Byte transmitted during a write transaction.
0x10	RX_DATA	RO	0x0000_0000	Byte captured during a read transaction.
0x14	CLK_DIV	R/W	0x0000_00FA	Clock-divider modulus. Default 250 gives 100 kHz SCL from 50 MHz.
0x18	INT_STATUS	RO	0x0000_0000	Event register mirroring DONE and ACK_ERROR.
0x1C	VERSION	RO	0x0001_0000	Constant IP version identifier.
other	—	—	—	Undecoded. Reads return 0; PSLVERR asserted.

Table 6 — APB register map.

4.4 Register bit definitions

4.4.1 CTRL — 0x00

Bits	Name	Access	Description
[31:2]	RESERVED	RO	Reads as zero. Writes ignored.
[1]	RW	R/W	Transaction direction: 0 = WRITE, 1 = READ. Persistent between transactions.
[0]	START	W1P	Write 1 to launch a transaction. Generates a one-PCLK internal pulse and is never stored, so it always reads back as 0. A write of 1 while the master is busy is discarded, which prevents a transaction from being restarted mid-flight.

Table 7 — CTRL register. W1P = write-1-to-pulse.

4.4.2 STATUS — 0x04

Bits	Name	Type	Description
[31:5]	RESERVED	—	Reads as zero.
[4]	I2C_BUS_BUSY	live	Bus-level activity; driven from master_busy at the top level.
[3]	BUS_ERROR	live	Master is in, or has entered, its ERROR state.
[2]	ACK_ERROR	sticky	Latched from master_ack_error on master_done. Cleared by writing CTRL.START = 1.
[1]	DONE	sticky	Set when master_done pulses. Cleared by writing CTRL.START = 1. This is the bit software polls.
[0]	BUSY	live	Direct reflection of master_busy.

Table 8 — STATUS register.

The distinction between live and sticky bits matters for software. BUSY tracks the master in real time and will already be LOW by the time a polling loop observes DONE. DONE and ACK_ERROR are latched precisely so that a slow poller cannot miss a completed transaction; they persist until the next transaction is launched.

4.4.3 Remaining registers

Register	Field	Notes
SLAVE_ADDR	[6:0]	Seven-bit address. The R/W bit is appended by the master, not stored here. Upper bits read as zero.
TX_DATA	[7:0]	Readable back for diagnostics. Sampled by the master at the address-ACK boundary, not at launch, so it may be updated up to that point.
RX_DATA	[7:0]	Updated only on master_done. Holds its value until the next transaction completes, so it may be read at leisure.
CLK_DIV	[31:0]	A write of zero is rejected and the previous value retained, because a modulus of zero would stall the tick generator permanently.
INT_STATUS	[1:0]	Bit 0 = DONE event, bit 1 = ACK_ERROR event. A single-read convenience; no interrupt output pin is implemented.
VERSION	[31:0]	Constant 0x0001_0000. Reading this register is the recommended presence check during bring-up.

Table 9 — SLAVE_ADDR, TX_DATA, RX_DATA, CLK_DIV, INT_STATUS and VERSION.

4.5 Software usage sequence

A write transaction: program SLAVE_ADDR and TX_DATA, clear CTRL.RW, then write CTRL.START = 1. Poll STATUS until DONE is set, then check ACK_ERROR. A read transaction is identical except that CTRL.RW is set and the captured byte is taken from RX_DATA once DONE is observed.

5 Clocking, Reset and Timing

5.1 Clock domain

The design is single-clock. The APB registers, the I²C master and the I²C slave are all clocked by PCLK at 50 MHz. Introducing a separate I²C clock domain would require synchronisers on every control and status path and would add clock-domain-crossing risk for no functional benefit, since the I²C bit rate is four hundred times slower than the system clock.

The slave nonetheless treats SCL and SDA as asynchronous inputs and synchronises them, because in a real system those lines originate off-chip.

5.2 Clock-enable generation

SCL is not produced by a divided clock. The divider instead produces a single-cycle clock enable, `i2c_tick`, and every I²C state transition and every SCL edge is gated by it. This is the correct approach for synthesis: one clock tree, no generated clocks to constrain, and no gated-clock cells.

```
always @(posedge clk) begin
    if (!rst_n) begin
        clk_count <= 32'd0;
        i2c_tick <= 1'b0;
    end
    else if (clk_count == clk_div - 1) begin
        clk_count <= 32'd0;
        i2c_tick <= 1'b1; // single-cycle enable
    end
    else begin
        clk_count <= clk_count + 1'b1;
        i2c_tick <= 1'b0;
    end
end
```

Listing 5.1 — Tick generator. `i2c_tick` is HIGH for exactly one PCLK cycle every `clk_div` cycles.

5.3 Frequency derivation

Each SCL half-period consumes one tick: the FSM releases SCL on one tick and pulls it low on the next, so the tick rate must be twice the target SCL frequency.

Quantity	Symbol	Expression	Value
System clock	<code>f_clk</code>	—	50.000 MHz
System clock period	<code>T_clk</code>	$1 / f_clk$	20 ns
Divider modulus	<code>clk_div</code>	programmable, default 250	250
Tick rate	<code>f_tick</code>	f_clk / clk_div	200.000 kHz
Tick period	<code>T_tick</code>	$clk_div \times T_clk$	5.000 μ s
SCL half-period	<code>t_HIGH</code> , <code>t_LOW</code>	<code>T_tick</code>	5.000 μ s
SCL frequency	<code>f_SCL</code>	$f_clk / (2 \times clk_div)$	100.000 kHz
Bit period	<code>T_bit</code>	$1 / f_SCL$	10.000 μ s
Byte + ACK duration	$9 \times T_bit$	—	90.000 μ s

Table 10 — Timing derivation for the default configuration.

The general expression is $\text{clk_div} = f_{\text{clk}} / (2 \times f_{\text{SCL}})$. For 50 MHz and 100 kHz this gives 250, the reset default. Because the value is programmable at runtime, the same RTL supports other system frequencies without recompilation — a 100 MHz clock simply needs $\text{clk_div} = 500$.

Simulation confirms the figures. In the master transcript the FSM enters SEND_ADDR at 10.21 μs and the slave reports the address byte matched at 90.21 μs : exactly 80 μs , or eight bit periods of 10 μs each.

5.4 Standard-mode timing compliance

All Standard-mode minimum timings are satisfied with substantial margin, because a 5 μs half-period is roughly an order of magnitude longer than the tightest requirement.

Parameter	Description	Std-mode min	Implemented	Margin
f_SCL	SCL clock frequency	$\leq 100 \text{ kHz}$	100 kHz	At limit
t_LOW	SCL LOW period	4.7 μs	5.0 μs	+0.3 μs
t_HIGH	SCL HIGH period	4.0 μs	5.0 μs	+1.0 μs
t_HD;STA	START hold time	4.0 μs	5.0 μs	+1.0 μs
t_SU;STO	STOP setup time	4.0 μs	5.0 μs	+1.0 μs
t_BUF	Bus free time between STOP and START	4.7 μs	$\geq 5.0 \mu\text{s}$	+0.3 μs
t_SU;DAT	Data setup time	250 ns	$\approx 5.0 \mu\text{s}$	large
t_HD;DAT	Data hold time	0 ns	$\approx 5.0 \mu\text{s}$	large

Table 11 — Standard-mode timing compliance. Values are nominal simulation figures.

Note on START hold time

The START state is deliberately split into two tick-gated phases. The first tick pulls SDA LOW while SCL is still released; the second pulls SCL LOW. This guarantees a full 5 μs START hold time and prevents the common failure mode in which SCL falls in the same cycle as SDA, producing a START that a slave with slow synchronisers may not see.

5.5 Reset strategy

Reset brings every register to a known value, returns all three FSMs to their idle states and releases both bus lines. Releasing the bus matters: a reset asserted mid-transaction must not leave SDA or SCL held LOW, because that would lock the bus for every other device on it.

Module	Reset signal	Polarity	Style	Reset state
apb_i2c_regs	PRESETn	Active LOW	Async assert, sync release	All registers zero; CLK_DIV = 250.
i2c_master	rst_n	Active LOW	Fully synchronous	State = IDLE; counters and shift registers zero; drive-low outputs deasserted.
i2c_slave	rst	Active HIGH	Fully synchronous	State = IDLE; synchronisers preset to bus idle (SDA = SCL = 1); sda_drive_low deasserted.

Table 12 — Reset behaviour per module, as currently implemented.

Finding R-1: inconsistent reset conventions

Three reset conventions are in use across three modules: asynchronous active-low, synchronous active-low, and synchronous active-high. The top level therefore has to invert PRESETn to produce an active-high rst for the slave while passing PRESETn directly to the master.

Recommendation: standardise on asynchronous-assert / synchronous-release, active-low, across all three modules. This is the convention already used by the APB block and is standard FPGA practice.

5.6 Slave synchroniser latency

The slave passes SCL and SDA through two flip-flops before use and keeps a third delayed copy for edge detection. This costs three PCLK cycles — 60 ns — between a bus transition and the slave reacting to it. Against a 5 µs half-period that is 1.2 % of a bit cell and therefore negligible, but it is the reason the slave must never be asked to respond within the same bit cell in which an edge occurs.

The synchronisers are reset to the bus idle level rather than to zero. Resetting them to zero would synthesise a spurious falling edge on both lines immediately after reset release, which the edge detectors would read as bus activity.

6 I²C Master Design

6.1 Responsibilities

The master owns the bus. It generates SCL, creates the START and STOP conditions, transmits the address and R/W bit, samples the acknowledge bits, moves one data byte in either direction, and reports the outcome. Once launched it requires no further input until it completes.

6.2 FSM state diagram

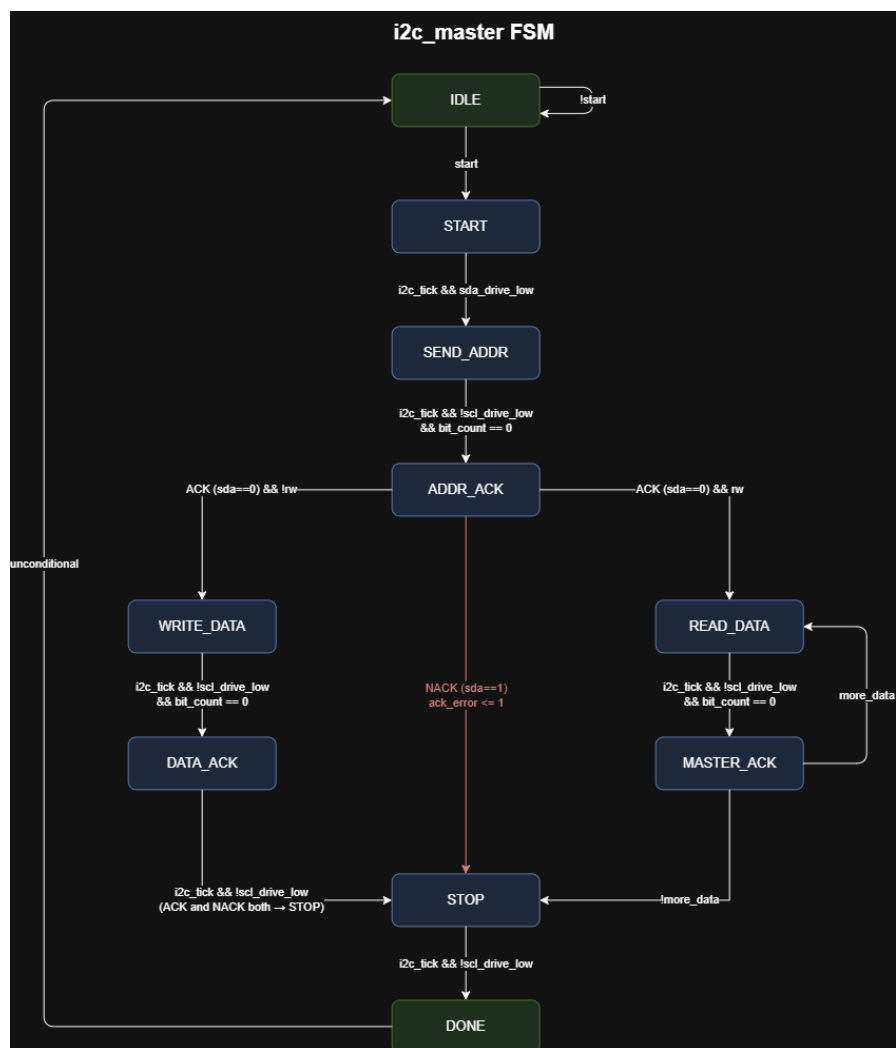


Figure 2 — I²C master FSM. Transitions are gated by `i2c_tick`; the highlighted path is the NACK abort route.

6.3 State table

State	Enc	Actions on entry / while resident	Exit condition	Next
IDLE	0	Deassert busy and done; release SDA and SCL. On start: assert busy, clear errors, load {slave_addr, rw} into tx_shift, set bit_count = 7.	start	START
START	1	Phase 1: on tick, pull SDA LOW while SCL is released — this is the START condition. Phase 2: on the next tick, pull SCL LOW.	tick & sda_drive_low	SEND_ADDR
SEND_ADDR	2	While SCL is LOW, place tx_shift[bit_count] on SDA. On tick, release SCL. While SCL is HIGH, on tick, pull SCL LOW and decrement bit_count.	tick & SCL HIGH & bit_count = 0	ADDR_ACK
ADDR_ACK	3	Release SDA so the slave may respond, then clock one bit cell. On the SCL-HIGH tick, sample SDA: HIGH sets ack_error; LOW reloads bit_count = 7 and loads tx_data (write) or clears rx_shift (read).	tick & SCL HIGH	STOP if NACK; READ_DATA if rw; else WRITE_DATA
WRITE_DATA	4	Identical bit-shifting mechanism to SEND_ADDR, using the freshly loaded tx_data.	tick & SCL HIGH & bit_count = 0	DATA_ACK
DATA_ACK	5	Release SDA; clock one bit cell; sample the slave acknowledge. A NACK sets ack_error but the transaction still terminates cleanly.	tick & SCL HIGH	STOP
READ_DATA	6	Release SDA for the whole byte. On each SCL-HIGH tick, capture SDA into rx_shift[bit_count] and decrement.	tick & SCL HIGH & bit_count = 0	MASTER_ACK
MASTER_ACK	7	Transfer rx_shift to rx_data. Leave SDA released — this transmits a NACK, the correct terminator for a single-byte read. Clock one bit cell.	tick & SCL HIGH	STOP
STOP	8	While SCL is LOW, pull SDA LOW. On tick, release SCL; on the next tick release SDA while SCL is HIGH — this is the STOP condition.	tick & SCL HIGH	DONE
DONE	9	Deassert busy; assert done for one cycle.	unconditional	IDLE
ERROR	10	Deassert busy; assert done and bus_error; release both lines.	unconditional	IDLE

Table 13 — I²C master FSM state table.

6.4 Acknowledge handling

All three acknowledge cells — address ACK, data ACK and master ACK — are structurally identical bit cells. The differences are in who drives and what a sampled HIGH means.

Cell	Master SDA	Slave SDA	Meaning of SDA sampled HIGH
ADDR_ACK	Released	Drives LOW if the address matches	No slave at this address. ack_error is set and the FSM diverts straight to STOP, aborting safely rather than pushing data onto a bus nobody is listening to.
DATA_ACK	Released	Drives LOW to accept the byte	Slave did not accept the byte. ack_error is set; the FSM proceeds to STOP regardless, so the bus is always left clean.

Cell	Master SDA	Slave SDA	Meaning of SDA sampled HIGH
MASTER_ACK	Released (= NACK)	Released	Not applicable — the master is the transmitter here. Releasing SDA transmits a NACK, telling the slave no further bytes are wanted.

Table 14 — Acknowledge cell behaviour.

The single-byte NACK in MASTER_ACK is intentional and protocol-correct: a master receiving the last byte of a transfer signals completion by not acknowledging it. Test 7 of the master testbench exists specifically to confirm this, and the slave transcript records a master NACK on every read.

6.5 Status signal semantics

Because DONE transitions unconditionally to IDLE, done is asserted for precisely one PCLK cycle. This satisfies the single-cycle completion-pulse requirement and is why the APB block latches it into a sticky status bit rather than exposing it directly.

7 I²C Slave Design

7.1 Responsibilities

The slave is a pure responder. It generates no clock, never drives SCL, and influences the bus only by pulling SDA LOW. It watches for START, receives the address byte, compares it against its parameterised address, and either participates in the transaction or ignores it until STOP.

7.2 Bus observation and condition detection

Because SCL and SDA are asynchronous to PCLK, both are passed through a two-flop synchroniser, and a third flop provides the delayed copy used for edge detection. The four derived signals are computed combinationally.

```
scl_rising  = scl_sync & ~scl_sync_d;
scl_falling = ~scl_sync & scl_sync_d;

// START: SDA falls while SCL is stable HIGH
start_cond  = ~sda_sync & sda_sync_d & scl_sync & scl_sync_d;

// STOP: SDA rises while SCL is stable HIGH
stop_cond   = sda_sync & ~sda_sync_d & scl_sync & scl_sync_d;
```

Listing 7.1 — Edge and bus-condition detection.

Requiring both scl_sync and scl_sync_d to be HIGH — rather than testing the current level alone — is what makes detection robust. It ensures SCL was already stable HIGH before the SDA transition, so a simultaneous SCL and SDA change cannot be misread as a START or STOP.

7.3 Priority structure

The slave state register is written by a four-level priority chain, and this structure is the single most important design decision in the module.

Priority	Condition	Effect
1 (highest)	rst	Full reset. All registers cleared; synchronisers preset to bus idle.
2	start_cond	Unconditional jump to RECV_ADDR from any state. Clears bit_count, rx_shift and all done/seen flags; releases SDA.
3	stop_cond	Unconditional jump to IDLE from any state. Releases SDA; clears the flags.
4 (lowest)	case (state)	Normal per-state behaviour.

Table 15 — Slave state-update priority.

Placing START and STOP above the state machine means the slave can never be stranded. A master that aborts mid-byte, a reset on the master side, a glitch — any of these produce a STOP or a fresh START, and the slave resynchronises immediately. Decoding START and STOP inside each state instead multiplies the logic and invariably misses a case. Test 18 of the slave testbench exercises exactly this path: a transaction is aborted mid-flight and the slave is confirmed to release SDA and return to IDLE.

7.4 FSM state diagram

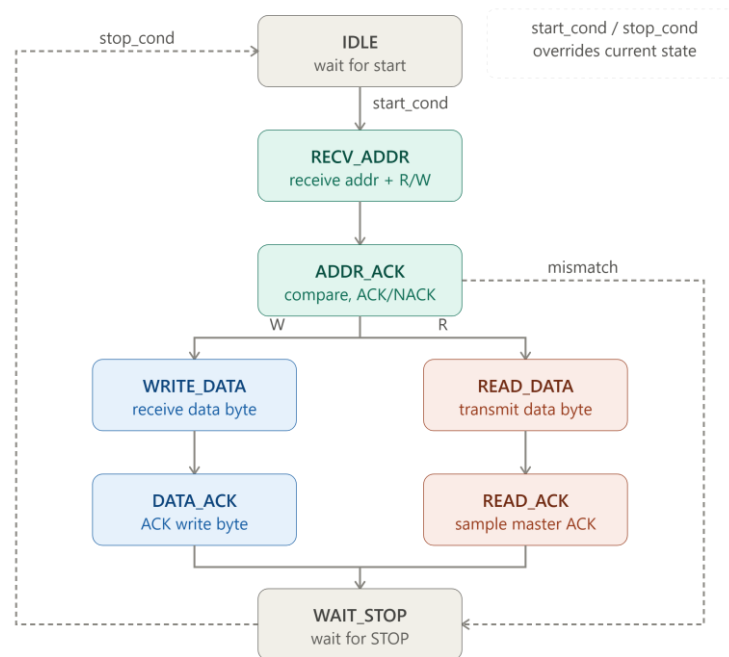


Figure 3 — I²C slave FSM, high-level view. The dashed paths are the global START/STOP overrides.

The detailed flow is reproduced below in four panels, split at the natural breaks in the original drawing. Together they show every edge condition and every register update in the slave.

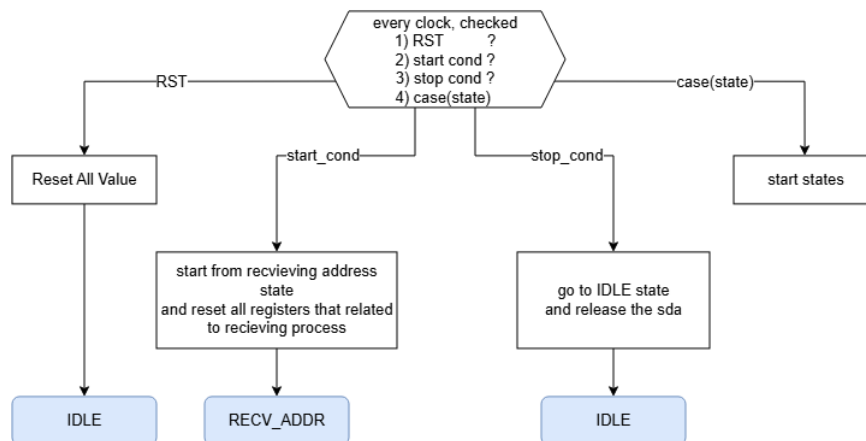


Figure 4 — Panel 1 — top-level priority: reset, START, STOP, then the state case.

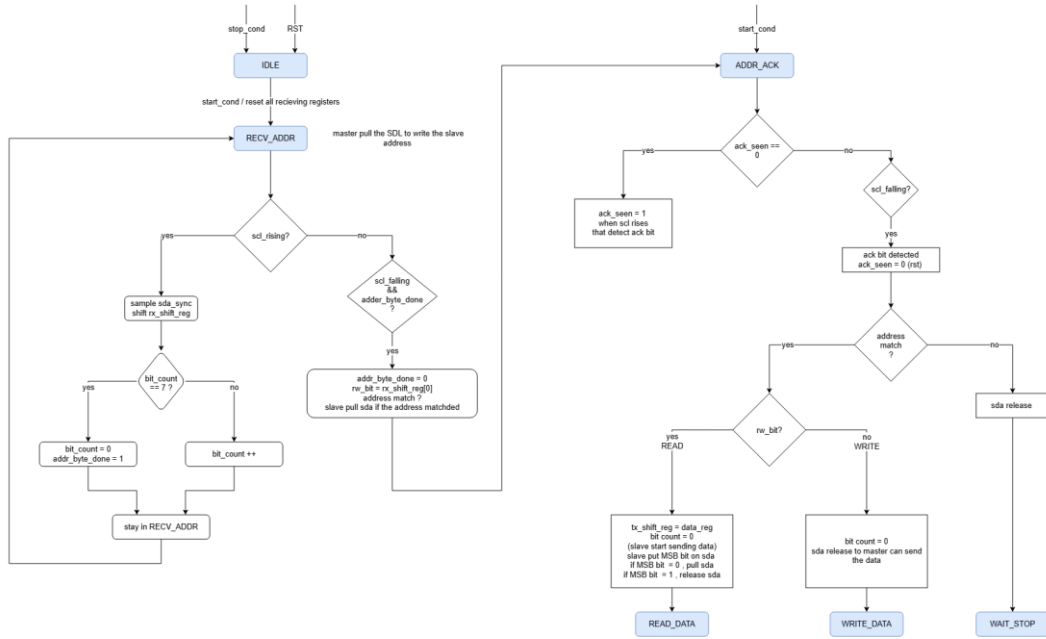


Figure 5 — Panel 2 — RECV_ADDR bit reception and the ADDR_ACK decision tree.

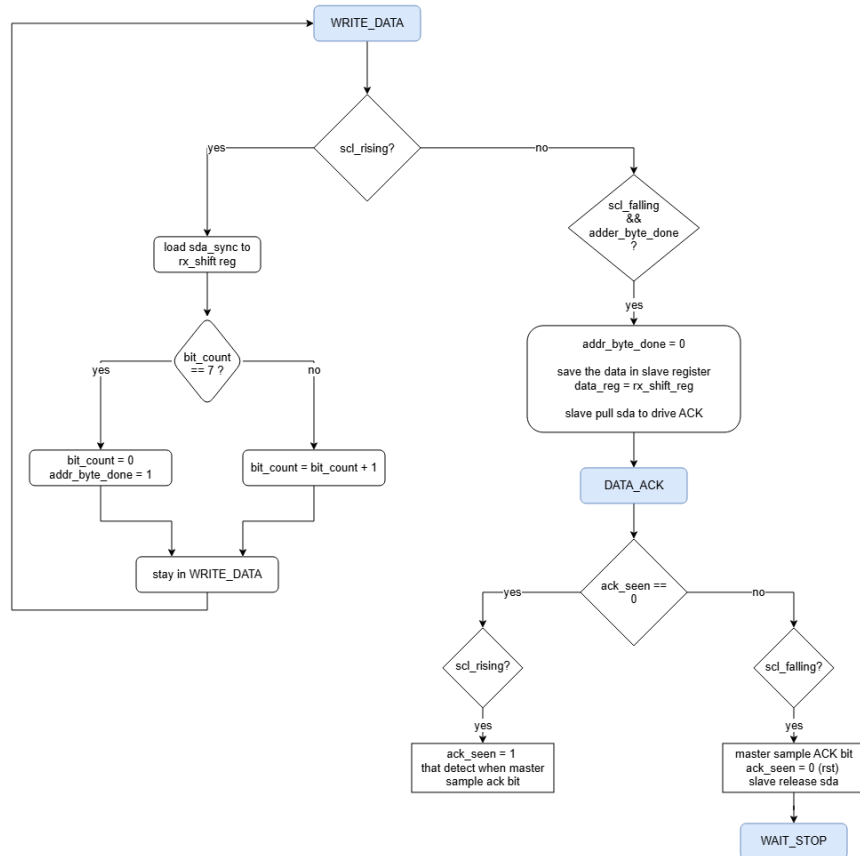


Figure 6 — Panel 3 — WRITE_DATA reception and the DATA_ACK cell.

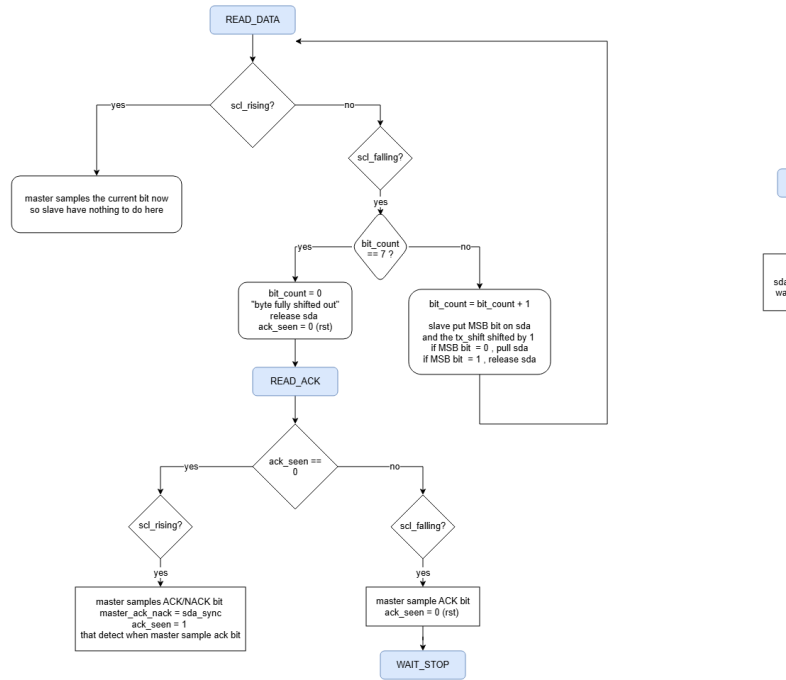


Figure 7 — Panel 4 — READ_DATA transmission, READ_ACK sampling and WAIT_STOP.

7.5 State table

State	Enc	Behaviour	Exit condition	Next
IDLE	0	Release SDA and wait. Entry to the machine is via the global start_cond override.	start_cond (global)	RECV_ADDR
RECV_ADDR	1	On each SCL rising edge, shift sda_sync into rx_shift_reg and increment bit_count. After the eighth bit, set addr_byte_done.	SCL falling & addr_byte_done	ADDR_ACK
ADDR_ACK	2	On the falling edge that ends the address byte, latch rw_bit and address_match and drive SDA LOW if the address matched. Wait for the master to sample on the rising edge, then act on the following falling edge.	SCL falling after ack_seen	READ_DATA if match & R; WRITE_DATA if match & W; WAIT_STOP if mismatch
WRITE_DATA	3	Shift eight bits in on rising edges. On the closing falling edge, commit rx_shift_reg to data_reg and drive SDA LOW to acknowledge.	SCL falling & data_byte_done	DATA_ACK
DATA_ACK	4	Hold SDA LOW across the master's sampling edge, then release on the next falling edge.	SCL falling after ack_seen	WAIT_STOP
READ_DATA	5	Drive the MSB from ADDR_ACK onward. On each falling edge, shift tx_shift_reg left and present the next bit. Do nothing on rising edges — the master is sampling and SDA must not move.	SCL falling & bit_count = 7	READ_ACK
READ_ACK	6	Release SDA. Capture the master's ACK/NACK on the rising edge into master_ack_nack for observability.	SCL falling after ack_seen	WAIT_STOP
WAIT_STOP	7	Release SDA and take no further part in the transaction.	stop_cond (global)	IDLE

Table 16 — I²C slave FSM state table. Encodings match the numeric states printed in the simulation transcript.

7.6 The ack_seen mechanism

Every acknowledge cell in the slave spans two SCL edges: a rising edge on which the master samples, and a falling edge on which it becomes safe for the slave to change SDA. The `ack_seen` flag sequences these two edges.

```
if (!ack_seen) begin
    if (scl_rising) ack_seen <= 1'b1;    // master is sampling now
end
else begin
    if (scl_falling) begin                // now safe to move SDA
        ack_seen <= 1'b0;
        <state transition and SDA update>
    end
end
```

Listing 7.2 — The two-edge acknowledge handshake.

Without this flag the slave would have to change SDA on the same edge the master is sampling, violating the setup requirement and risking a false START or STOP. The same pattern appears identically in `ADDR_ACK`, `DATA_ACK` and `READ_ACK`, which keeps the three cells structurally consistent and easy to review.

7.7 Read-path bit presentation

The read path deserves particular attention because it is where most I²C slave implementations go wrong. The first bit is presented at the end of `ADDR_ACK`, not at the start of `READ_DATA`.

```
// in ADDR_ACK, on the falling edge, for a READ transaction:
tx_shift_reg <= data_reg;
bit_count    <= 3'd0;
sda_drive_low <= ~data_reg[7];    // MSB presented immediately
state        <= READ_DATA;
```

Listing 7.3 — Early presentation of the read MSB.

The master expects valid data on the very first SCL rising edge after the acknowledge cell. Waiting until `READ_DATA` to present the MSB would be one bit cell too late, and the master would capture seven correct bits and one wrong one. Test 4 of the slave testbench and Test 2 of the master testbench both verify the complete byte, which is what catches this class of off-by-one error.

8 Shared Bus and Open-Drain Behaviour

8.1 Electrical model

An I²C line is a wired-AND. Every device may pull the line LOW or release it, never drive it HIGH; a pull-up resistor restores the HIGH level once all devices have released. The line is therefore LOW if any device pulls it LOW, and HIGH only if every device has released.

Master drive_low	Slave drive_low	Bus level	Meaning
0	0	HIGH (pull-up)	Line released by both. Logic 1.
0	1	LOW	Slave asserting — for example, an ACK.
1	0	LOW	Master asserting — for example, transmitting a 0.
1	1	LOW	Both asserting. No contention: this is the defining property of open-drain.

Table 17 — Wired-AND truth table for SDA.

8.2 Implementation

The bus is formed at the top level by continuous assignments. Each participant either drives a hard zero or presents high impedance; the tri-state value 1'bz combined with the tri1 net type in the testbench models the pull-up resistor.

```
// In Top_Module
assign SDA = slave_sda_drive_low ? 1'b0 : 1'bz;

// In the testbench, the pull-up is modelled by the net type
tri1 SDA;
tri1 SCL;
```

Listing 8.1 — Open-drain bus formation at the top level.

The master contributes to SDA and SCL directly through its own inout ports, so the top level only has to add the slave's contribution to SDA. SCL has a single contributor because this is a single-master design without clock stretching; adding a slave SCL contribution later, to support stretching, would be a one-line change of the same shape.

9 Top-Level Integration — Top_Module

Top_Module is the synthesis boundary of the whole subsystem. It is deliberately thin: it contains no state machine and no arithmetic of its own. Its entire job is to expose the pins that leave the chip, instantiate the three functional blocks, wire them together with internal nets, and form the shared open-drain bus described in Chapter 8. Everything else — the register file, the protocol engine, the responder — lives one level down and is reached only through those nets.

This structure is what keeps the design portable. Because only the APB pins and the two I²C pins cross the boundary, the block can be dropped into any APB-based system without modification, and the internal wiring can be changed without affecting anything outside.

9.1 External interface

Exactly two interfaces leave the module: the APB4 slave port, driven by the processor or by the APB testbench, and the two-wire I²C bus. Both bus lines are declared inout because they are bidirectional open-drain nets.

Port	Direction	Width	Description
PCLK	input	1	System clock. Clocks all three internal blocks; 50 MHz nominal.
PRESETn	input	1	Active-low system reset.
PADDR	input	12	APB address bus; selects the register being accessed.
PSEL	input	1	APB select. Marks the SETUP phase of a transfer.
PENABLE	input	1	APB enable. Marks the ACCESS phase.
PWRITE	input	1	APB direction: 1 = write, 0 = read.
PWDATA	input	32	APB write data.
PRDATA	output	32	APB read data; zero outside an active read.
PREADY	output	1	APB ready. Tied HIGH — no wait states.
PSLVERR	output	1	APB slave error; asserted on an undecoded address.
SDA	inout	1	I ² C serial data line, open-drain.
SCL	inout	1	I ² C serial clock line, open-drain.

Table 18 — Top_Module port list.

9.2 Internal interconnect

The nets below are private to the top level. They form the control and status channel between the register file and the master, and they are the reason neither block needs to know anything about the other's protocol.

Net	Width	Direction	Purpose
master_start	1	Bridge → master	One-PCLK pulse generated by writing CTRL.START; launches a transaction.
master_rw	1	Bridge → master	Transaction direction taken from CTRL.RW.
master_slave_addr	7	Bridge → master	Target address from SLAVE_ADDR.
master_tx_data	8	Bridge → master	Byte to transmit, from TX_DATA.

Net	Width	Direction	Purpose
master_clk_div	32	Bridge → master	Divider modulus from CLK_DIV; sets the SCL frequency.
master_rx_data	8	Master → bridge	Byte captured on a read; latched into RX_DATA.
master_busy	1	Master → bridge	Transaction in progress; drives STATUS.BUSY.
master_done	1	Master → bridge	Single-cycle completion pulse; sets the sticky DONE bit.
master_ack_error	1	Master → bridge	No acknowledge received; sets the sticky ACK_ERROR bit.
master_bus_error	1	Master → bridge	Master entered its ERROR state.
i2c_bus_busy	1	internal	Bus-activity flag; in a single-master system it is simply master_busy.
slave_sda_drive_low	1	Slave → bus	The slave's only means of influencing the bus: pull SDA LOW.
slave_rx_data	8	Slave → (observation)	Byte received by the slave; used by the testbench to confirm a write.
rst	1	internal	Inverted PRESETn, supplying the slave's active-high reset.

Table 19 — Internal nets inside Top_Module.

9.3 Instantiation and bus formation

Three instances are created. The APB bridge is connected to the external APB pins on one side and the internal control/status nets on the other. The master receives those nets and drives SDA and SCL directly through its own bidirectional ports. The slave is parameterised with address 0x50, observes both lines as inputs, and contributes to SDA through a single drive-low control combined at the top level.

```

wire rst = !PRESETn;           // slave uses an active-high reset

// Open-drain contribution of the slave, and the bus-busy flag
assign SDA      = slave_sda_drive_low ? 1'b0 : 1'bz;
assign i2c_bus_busy = master_busy;

apb_i2c_regs Bridge ( .PCLK(PCLK), .PRESETn(PRESETn), .PADDR(PADDR),
                     .PSEL(PSEL), .PENABLE(PENABLE), .PWRITE(PWRITE),
                     .PWRITE(PWRITE), .PRDATA(PRDATA), .PREADY(PREADY),
                     .PSLVERR(PSLVERR), /* control and status nets */ );

i2c_master i2c_master ( .clk(PCLK), .rst_n(PRESETn), .start(master_start),
                      .rw(master_rw), .slave_addr(master_slave_addr),
                      .tx_data(master_tx_data), .rx_data(master_rx_data),
                      .clk_div(master_clk_div), .busy(master_busy),
                      .done(master_done), .ack_error(master_ack_error),
                      .bus_error(master_bus_error), .sda(SDA), .scl(SCL) );

i2c_slave #(SLAVE_ADDRESS(7'h50)) i2c_slave (
    .clk(PCLK), .rst(rst), .sda_in(SDA), .scl_in(SCL),
    .sda_drive_low(slave_sda_drive_low),
    .rx_data(slave_rx_data) );

```

Listing 9.1 — Top_Module: reset inversion, open-drain assignment and the three instantiations (condensed).

9.4 How a transaction flows through the top level

Reading the module from the outside in makes the data flow easy to follow. Software writes SLAVE_ADDR, TX_DATA and CTRL through the APB pins; the bridge decodes the write and raises master_start for one cycle. The master takes ownership of SDA and SCL and runs the full protocol sequence, gated by the tick derived from master_clk_div. The slave, watching the same two wires, detects the START, matches address 0x50, and either captures the byte or transmits one back by pulling SDA LOW at the right moments. When the master reaches DONE it pulses master_done; the bridge latches DONE, ACK_ERROR and, for a read, master_rx_data into RX_DATA. Software polling STATUS then sees the result.

Step	Where it happens	What occurs
1	APB pins → Bridge	Software programs SLAVE_ADDR, TX_DATA and CTRL; the bridge decodes the access.
2	Bridge → master	CTRL.START generates a one-cycle master_start pulse; direction and payload are already on the control nets.
3	Master → bus	START condition, address + R/W byte, acknowledge cell, data byte, STOP — all gated by the clock enable.
4	Bus → slave	The slave detects START, matches its address, and receives or transmits the data byte.
5	Master → bridge	master_done pulses; status and, for a read, the captured byte are latched into the register file.
6	Bridge → APB pins	Software reads STATUS and RX_DATA to confirm completion and retrieve the data.

Table 20 — Transaction flow through Top_Module.

Integration note

The top level currently inverts PRESETn for the slave while passing PRESETn unchanged to the master, because the two blocks were written with different reset conventions (Finding R-1, Section 5.5). Adopting a single active-low convention would remove the inverter and the asymmetry altogether.

slave_rx_data is brought to the top level but is not exposed on a port; it exists so that a testbench binding into the hierarchy can confirm a write without disturbing the bus.

10 Verification and Simulation Results

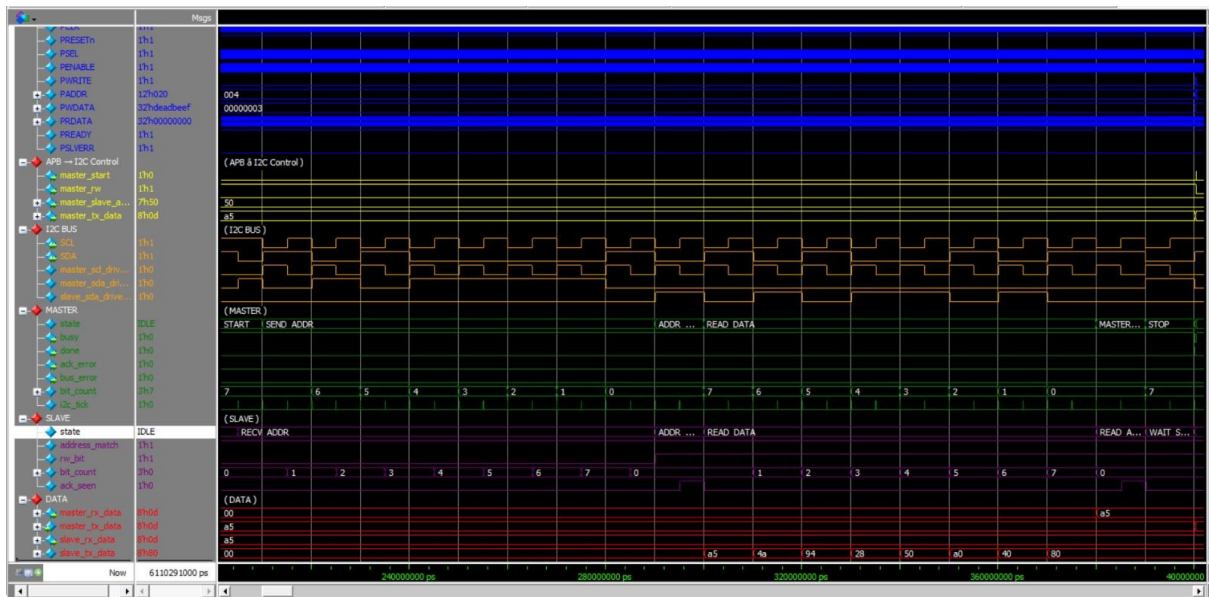
10.1 Strategy

Verification is layered. Each block is first exercised standalone by a self-checking testbench that prints an explicit PASS or FAIL line per check and a summary at the end, so a run is judged by its summary rather than by eye. The master and slave testbenches then meet on a shared open-drain bus, which turns every master transaction into an end-to-end check: the byte written is compared against the byte the slave stored, and the byte read is compared against the byte the slave transmitted.

Testbench	Target	Checks	Result
i2c_master_tb	i2c_master	14	14 passed, 0 failed
i2c_slave_tb	i2c_slave	22	22 passed, 0 failed
apb_i2c_tb	apb_i2c_regs	38 coverage bins	100 % functional coverage

Table 21 — Verification summary.

Top modules waveform:



10.2 APB register-file verification

The APB testbench models the SETUP and ACCESS phases through a `drive_apb()` task and predicts PRDATA and PSLVERR with a golden model. A constrained-random class distributes activity across all eight valid register offsets plus the invalid address range, and samples functional coverage for read/write combinations, I²C status inputs, zero clock-divider writes, and START attempts while the master is busy or idle.

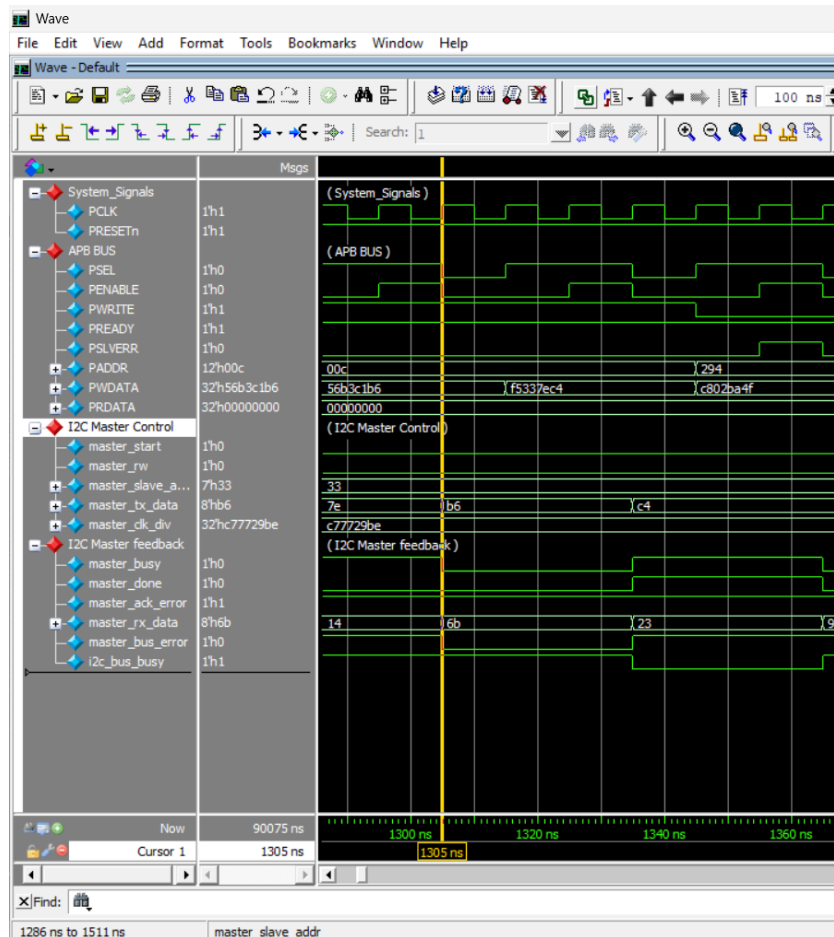


Figure 8 — APB write transaction: SETUP followed by ACCESS correctly writes the register.

Figure 10 — START command pulse: writing CTRL[0] = 1 generates exactly one PCLK cycle of master_start.

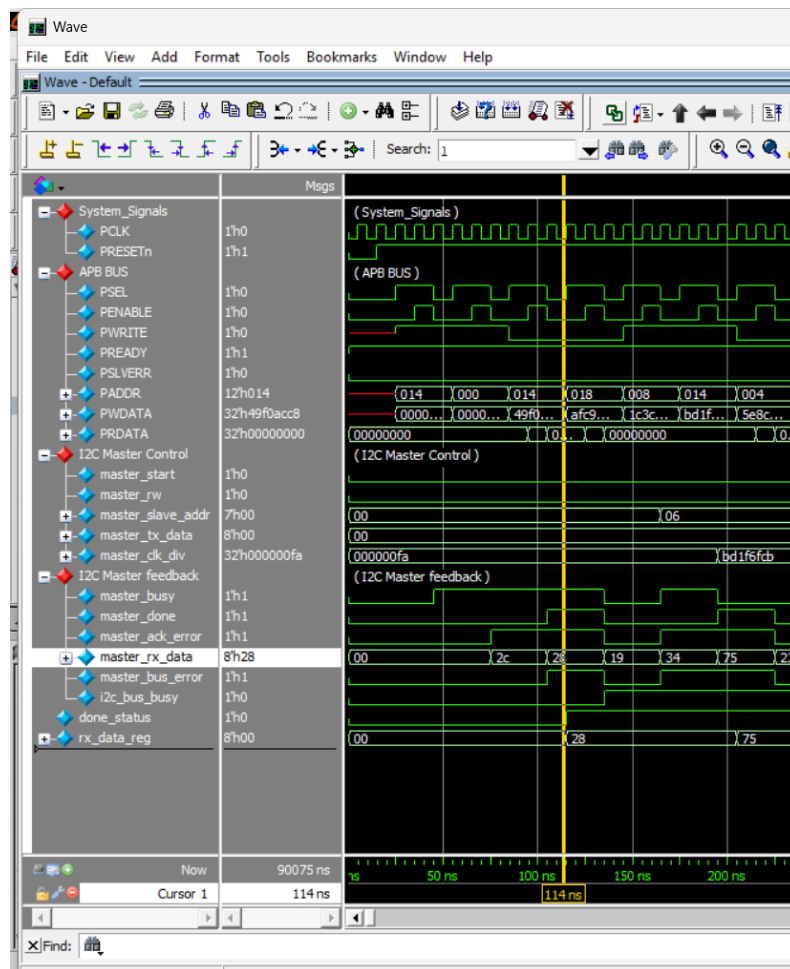


Figure 11 — Status and data feedback captured from the I²C master into the register block.

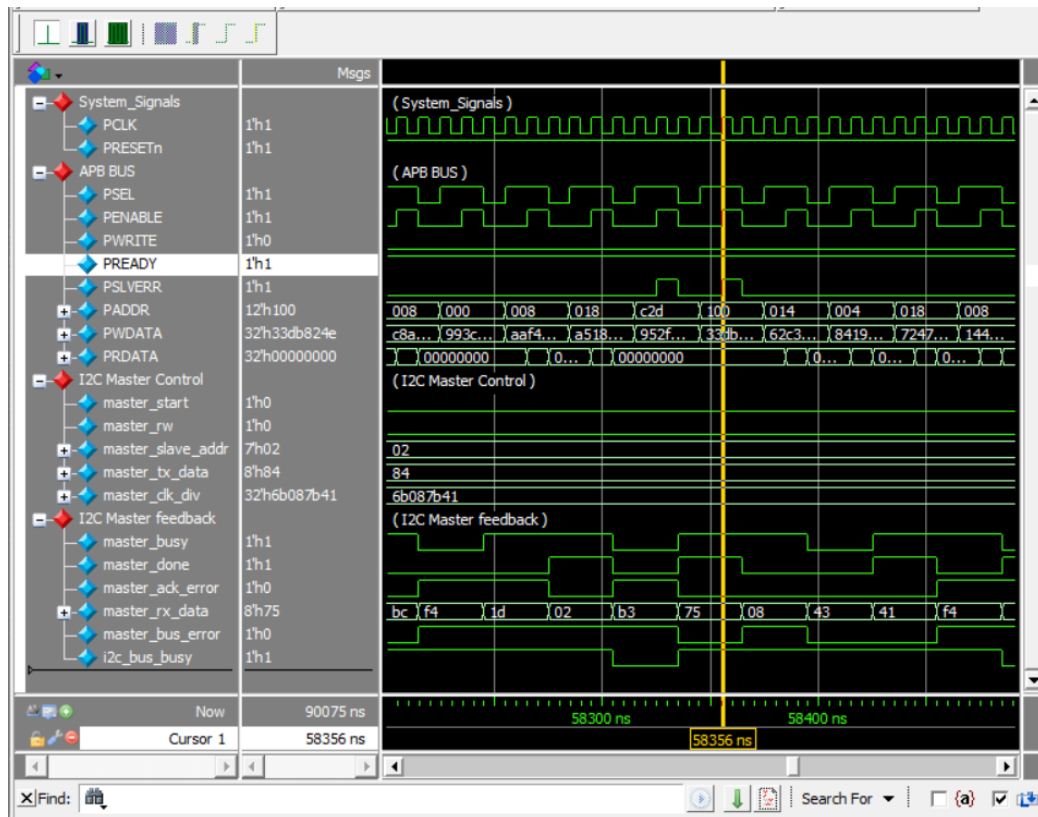


Figure 12 — An invalid APB address asserts PSLVERR during the ACCESS phase.

10.3 I²C master waveforms

The master testbench drives the design against the companion slave and records the following captures. Together they cover reset behaviour, both bus conditions, both transfer directions, bit ordering, and the unanswered-address abort path.

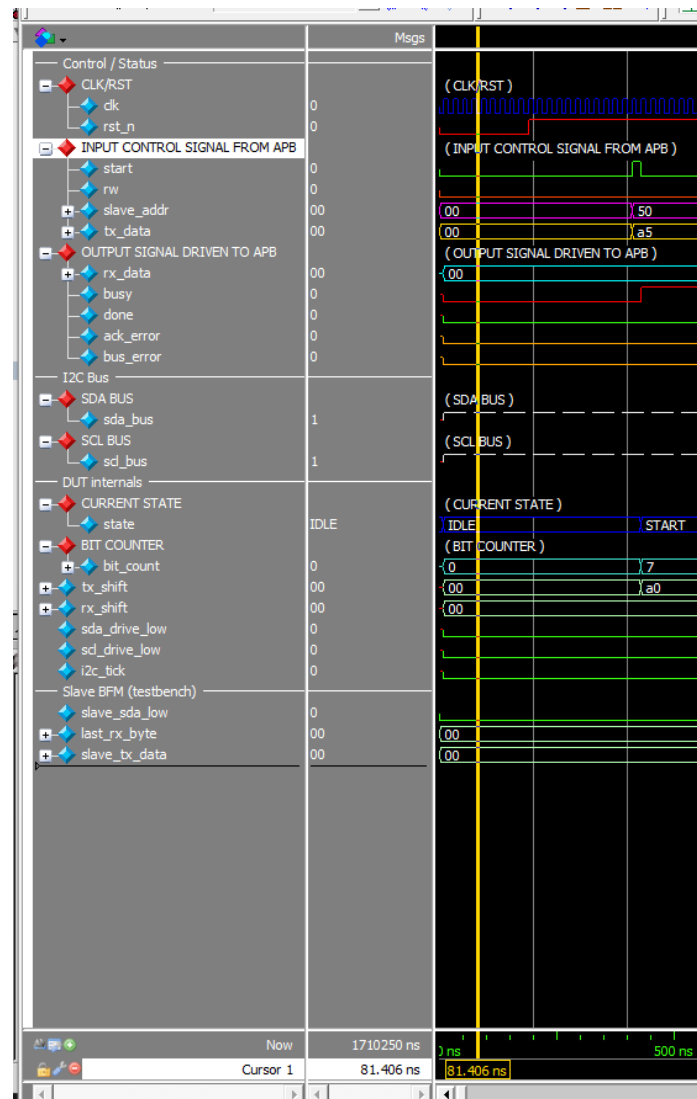


Figure 13 — Reset: APB and I²C-facing signals return to the idle/default condition.

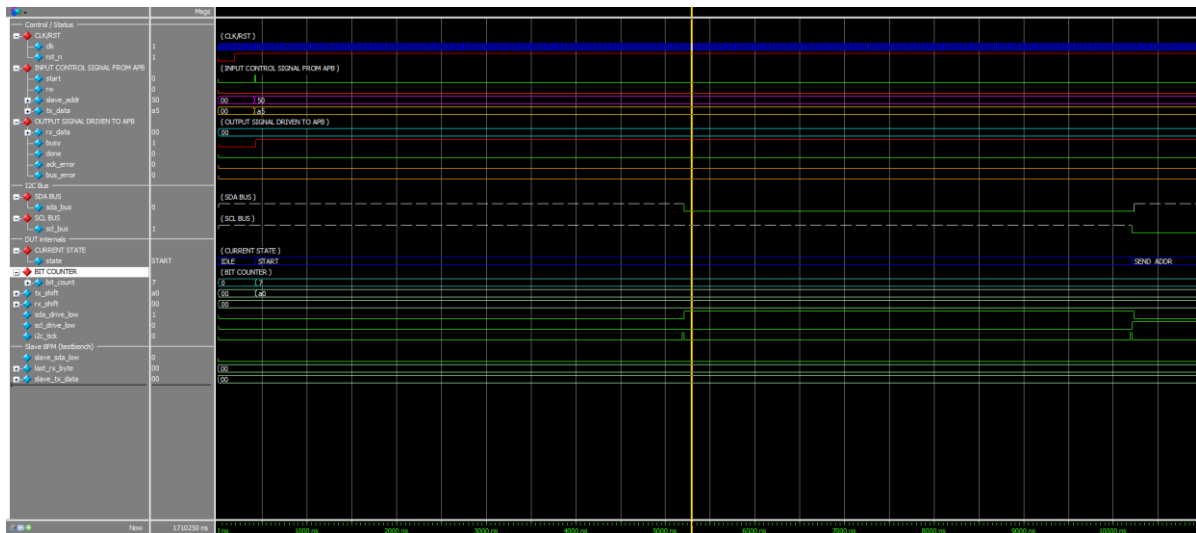


Figure 14 — START condition generated by the master.

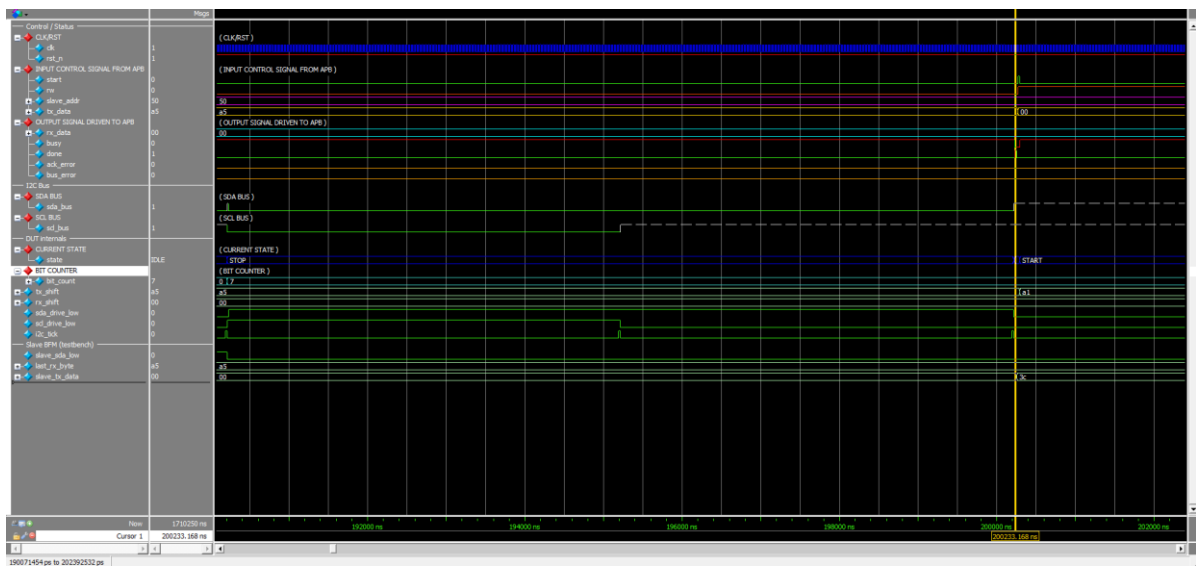


Figure 15 — STOP condition generated by the master.

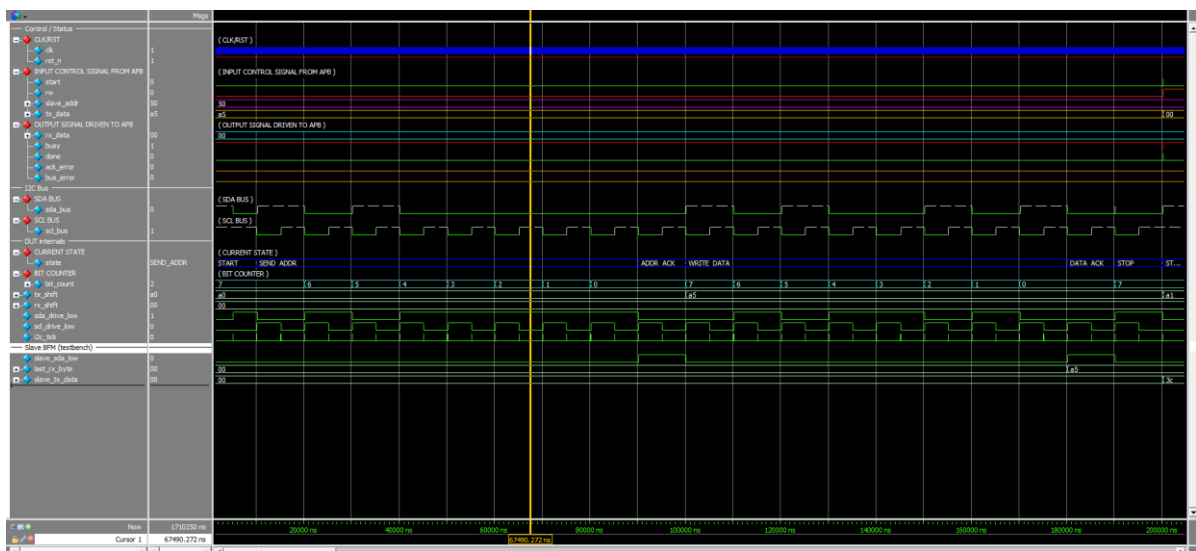


Figure 16 — Test 1 — WRITE: correct address and data transfer.

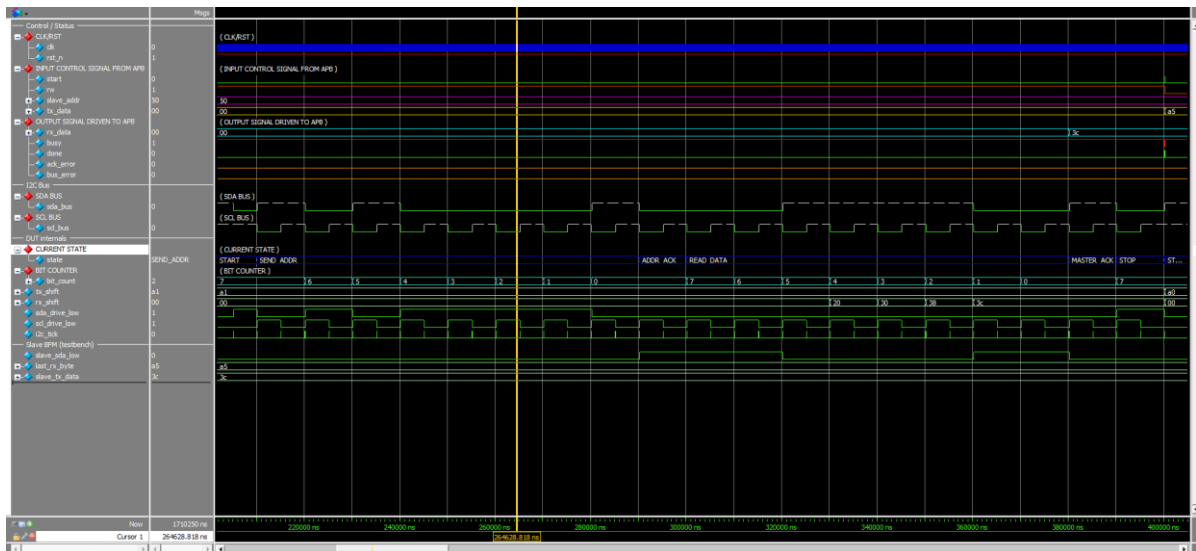


Figure 17 — Test 2 — READ: correct address and readback.

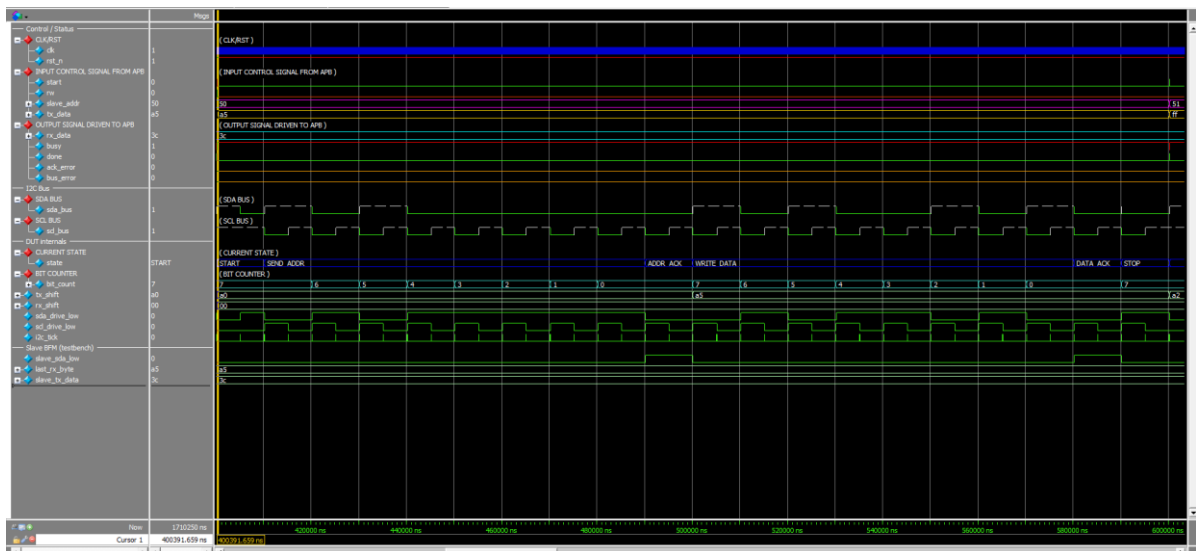


Figure 18 — Test 3 — WRITE: MSB-first bit ordering confirmed.

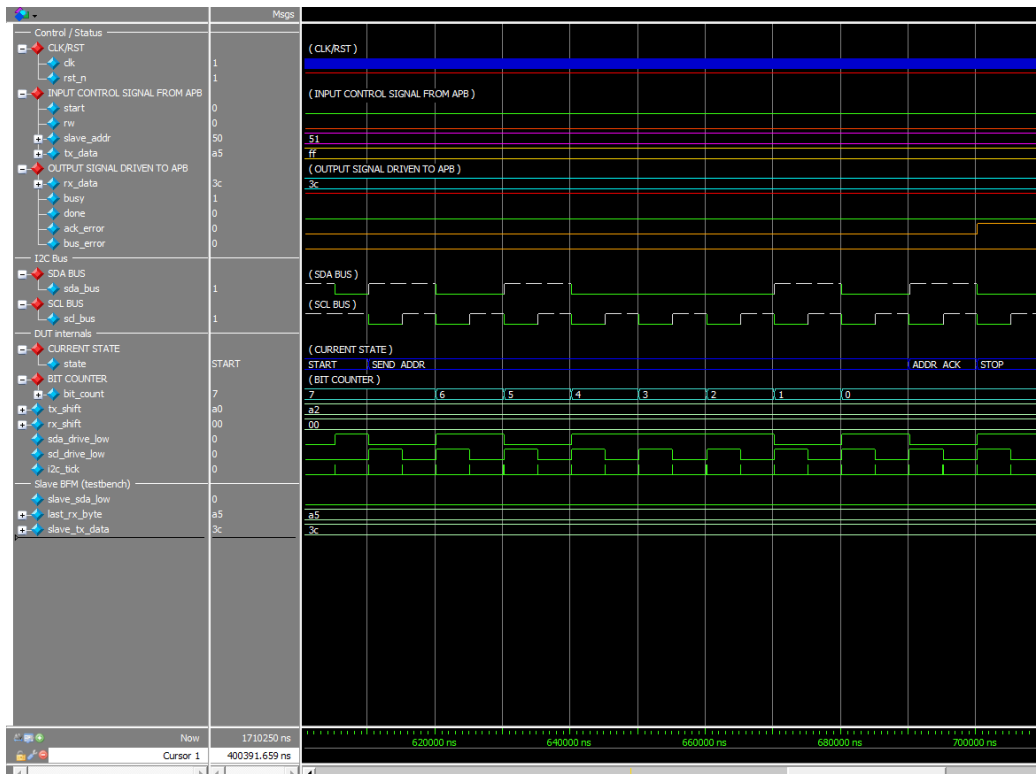


Figure 19 — Test 4 — WRITE to a wrong address: the expected NACK and clean abort.

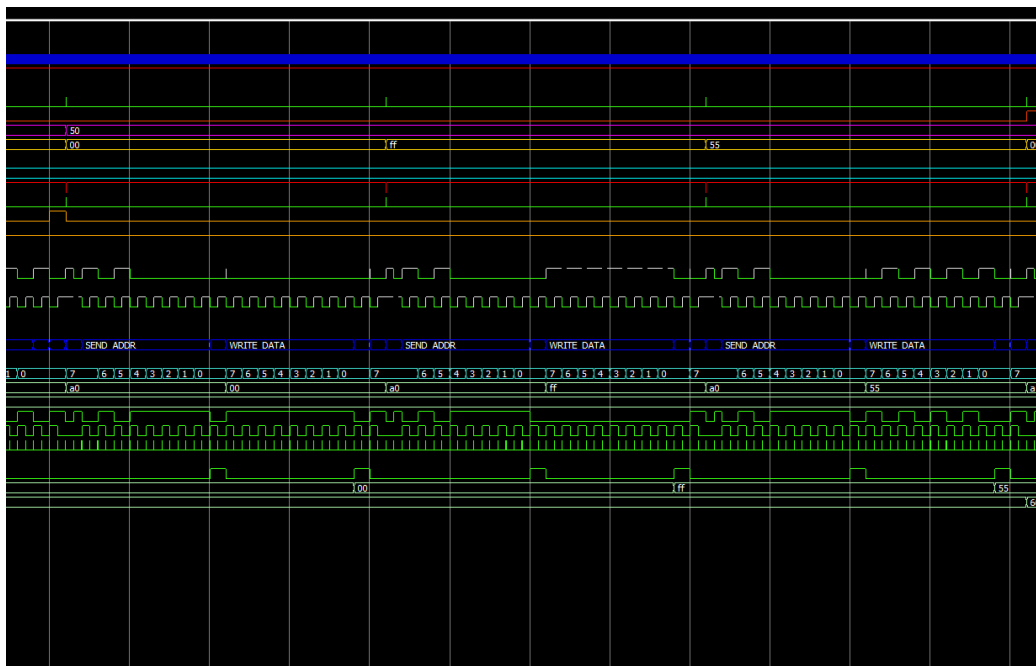


Figure 20 — Test 5 — multiple consecutive WRITE transactions.

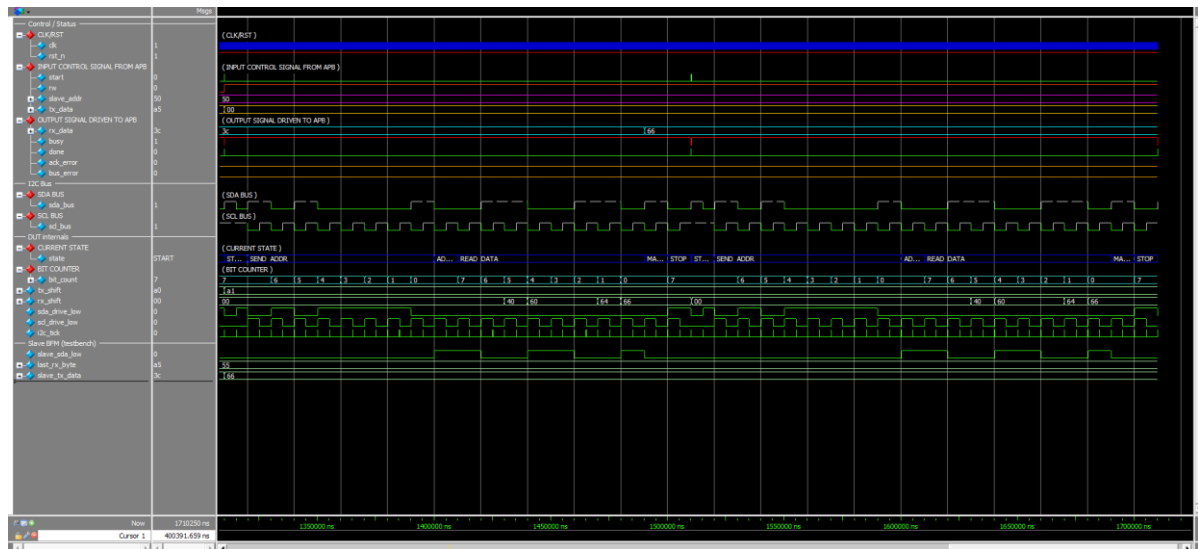


Figure 21 — Test 6 — multiple READ transactions returning a stable value.

10.4 I²C slave waveforms

The slave testbench applies bus stimulus directly and checks the responder's behaviour, including the abort path in which a transaction is terminated mid-flight.



Figure 22 — Slave testbench: START condition detected from the idle state.

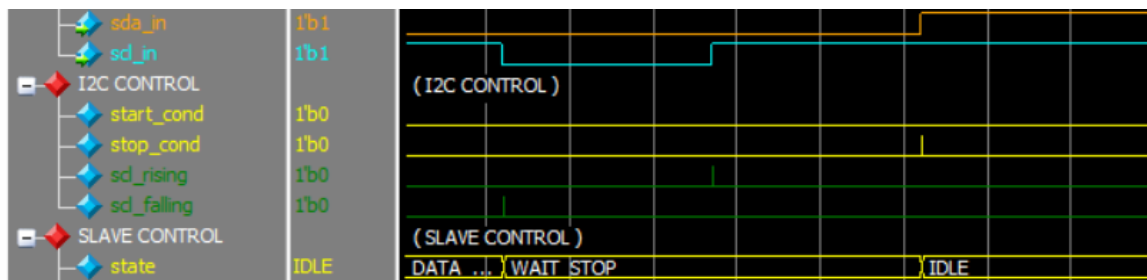


Figure 23 — Slave testbench: STOP condition returns the FSM to IDLE.

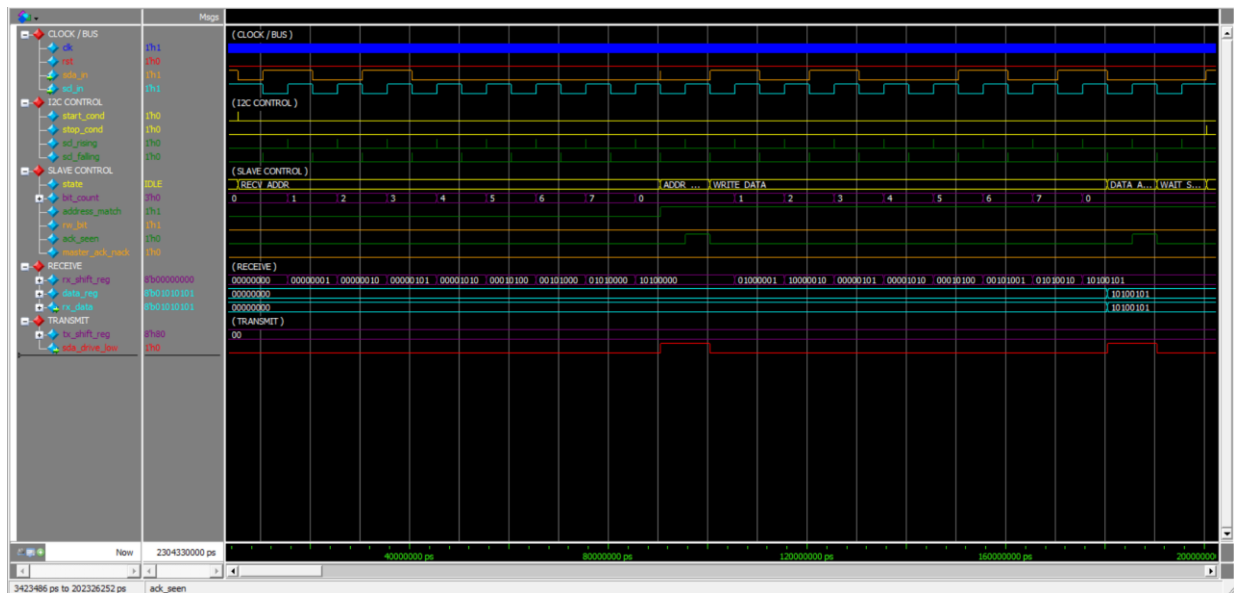


Figure 24 — Test 1 — WRITE: address matched, byte received and acknowledged.

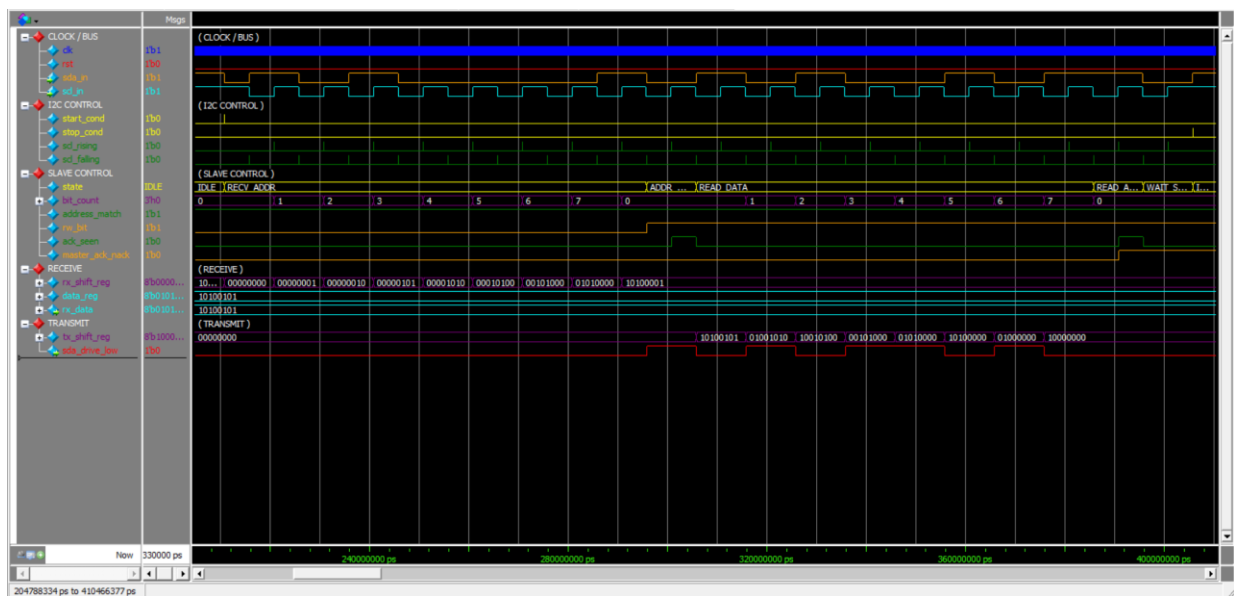


Figure 25 — Test 2 — READ: the stored byte is transmitted MSB-first.

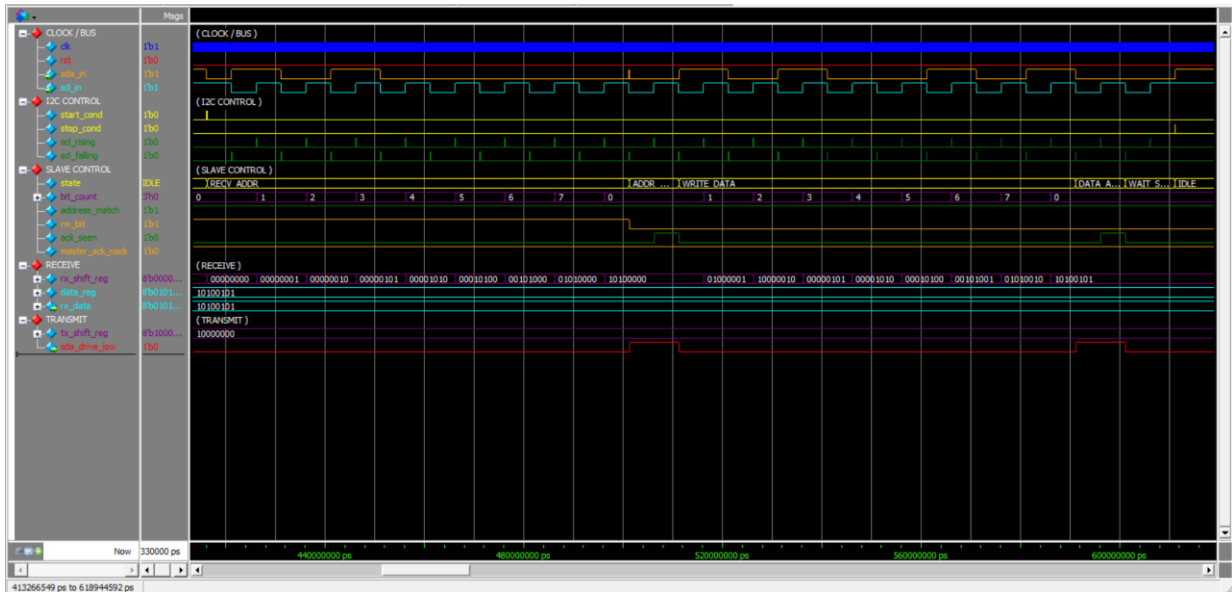


Figure 26 — Test 3 — WRITE with a new data pattern.



Figure 27 — Test 4 — wrong address: the slave ignores the transaction until STOP.

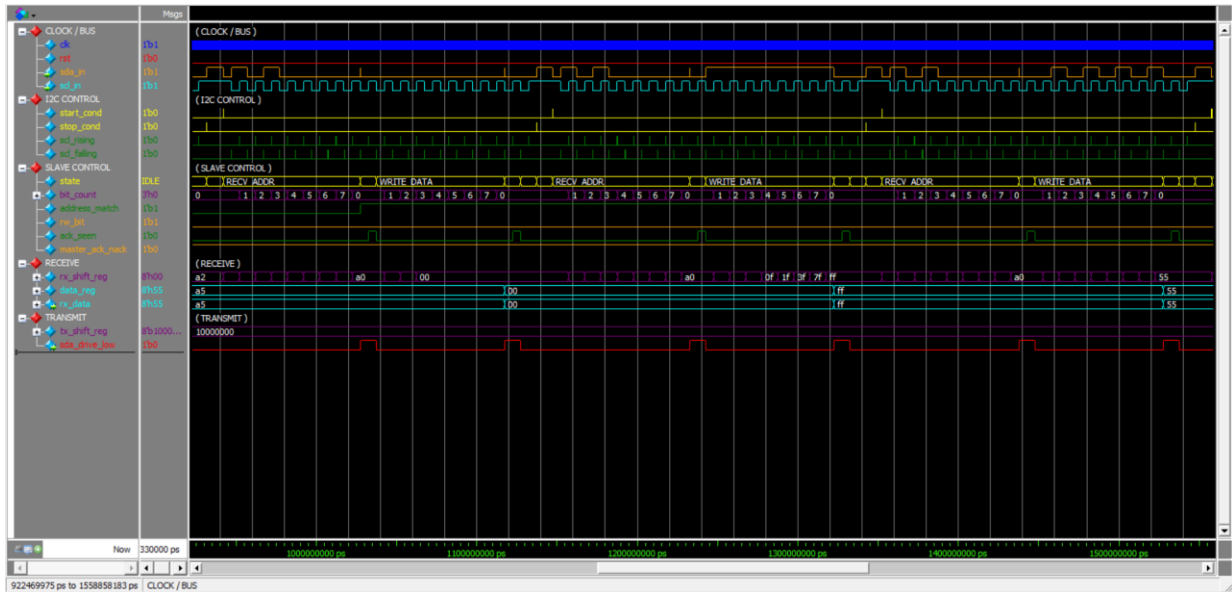


Figure 28 — Test 6 — multiple READ transactions.

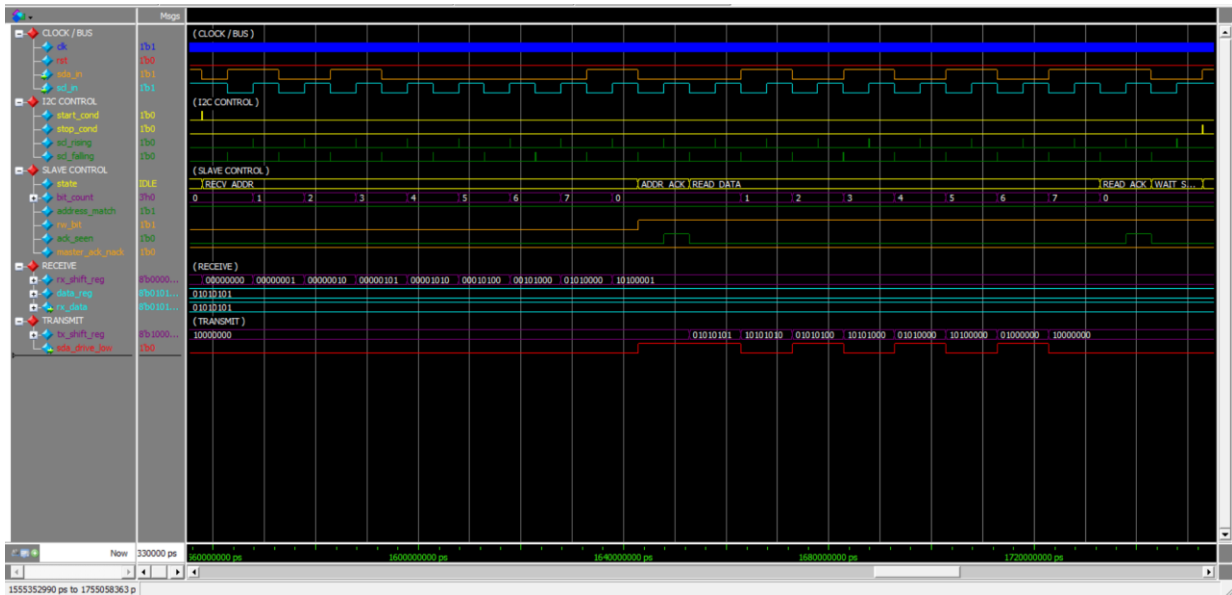


Figure 29 — Test 7 — second READ confirming the retained data register.

11 Coverage Analysis

11.1 Functional coverage

A single covergroup, `cvr_gp`, samples APB transactions. It contains seven coverpoints and one cross, totalling thirty-eight bins. All thirty-eight are hit.

Coverpoint	Bins	Hit	What it proves
<code>cp_addr</code>	9	9	Every one of the eight decoded offsets was accessed, and at least one undecoded address was exercised.
<code>cp_rw</code>	2	2	Both transfer directions were exercised.
<code>cp_done</code>	2	2	The DONE status bit was observed both set and clear.
<code>cp_ack_err</code>	2	2	The ACK_ERROR path was exercised, not merely the happy path.
<code>cp_bus_err</code>	2	2	The BUS_ERROR status bit was exercised at the register interface.
<code>cp_clk_div_zero</code>	1	1	The zero-divider rejection guard was hit.
<code>cp_start_while_busy</code>	2	2	START was written both while idle (accepted) and while busy (rejected).
<code>cx_addr_rw</code>	18	18	Every register was exercised in both directions, including writes to read-only registers.
TOTAL	38	38	100 % functional coverage.

Table 22 — Functional coverage summary for `apb_i2c_regs`.

The cross is the most informative item in the table. Reaching all eighteen bins means no register was tested in only one direction — a common gap that leaves, for instance, a broken read multiplexer on a write-mostly register undetected.

11.2 Code coverage

Metric	Bins	Hits	Misses	Coverage
Statement	38	38	0	100.00 %
Branch	38	38	0	100.00 %
Condition	5	5	0	100.00 %
Expression	8	8	0	100.00 %
Toggle	318	315	3	99.05 %
Overall (filtered)	—	—	—	99.81 %

Table 23 — Code coverage for `apb_i2c_regs`.

11.3 Analysis of the toggle gap

Three toggle bins out of three hundred and eighteen are unhit. Both causes are understood and neither represents a functional gap.

Node	1→0	0→1	Coverage	Explanation
PREADY	no	no	0 %	PREADY is a constant 1'b1 by design — every access completes without a wait state, so a toggle is impossible. The correct disposition is a documented waiver.
PRESETn	yes	no	50 %	Reset is asserted once at time zero and released, and is never re-asserted during the run. Adding a mid-simulation reset-recovery test would close this bin and is worthwhile on its own merits.

Table 24 — Toggle coverage exclusions.

Recommended actions: waive PREADY with the justification recorded above, and add a reset-recovery test to close the PRESETn bin. The latter is a genuine test-plan improvement rather than a coverage-chasing exercise, because it would also verify that an in-flight I²C transaction is cleanly abandoned when reset is asserted.

12 Future Enhancements

Ordered by value per unit of effort.

#	Enhancement	Effort	Benefit
1	Bus timeout and ERROR-state recovery	Low	Populates a currently unreachable state and makes the master robust against a hung bus. Should be done before any reuse.
2	Repeated START and multi-byte transfers	Medium	Enables the register-pointer read idiom that most real I ² C devices require. The single change that would make the master genuinely reusable.
3	Transmit and receive FIFOs	Medium	Decouples software from the bit rate and removes the need for tight polling.
4	Interrupt output with a mask register	Low	Removes polling entirely. INT_STATUS already exists; only the pin and mask logic are missing.
5	Clock stretching support	Medium	Required for interoperability with slow slaves. Needs SCL sensing in the master bit cell.
6	Fast mode (400 kHz)	Low	Mostly a matter of re-verifying the timing margins; the divider is already programmable.
7	Multi-master arbitration	High	Needed only for genuinely multi-master systems.
8	10-bit addressing	Medium	Needed only for devices that use it.
9	Formal property verification of the FSMs	Medium	Would prove deadlock-freedom and state reachability exhaustively.
10	UVM verification environment	High	Appropriate if this IP moves toward production; excessive for the present scope.

Table 25 — *Future enhancements, ordered by value per unit of effort.*

13 Conclusion

This project set out to build a clean, modular, synthesisable I²C master controlled through an AMBA APB4 register interface and verified in simulation against a companion I²C slave on a shared open-drain bus. The architecture, the register map, the two state machines and the bus model were all specified before any RTL was written, and the implementation follows that specification closely.

At block level the design works. The I²C master executes protocol-correct single-byte write and read transactions, generates valid START and STOP conditions, transmits MSB-first, samples acknowledges in the correct bit cell, aborts cleanly on an unanswered address rather than hanging, and terminates single-byte reads with the NACK the protocol requires; all fourteen checks in its testbench pass. The I²C slave detects START and STOP from any state, matches its address, receives and transmits bytes correctly, and recovers from aborted transactions; all twenty-two checks pass. The APB register file implements the specified map exactly, enforces a deterministic error policy, and reaches 100 % functional coverage across thirty-eight bins with 100 % statement, branch, condition and expression coverage.

Top_Module brings the three blocks together behind a single APB port and a single two-wire bus, with the open-drain behaviour formed by continuous assignment. The one piece of avoidable friction at this level is the reset convention: three modules use three different styles, so the top level has to invert PRESETn for the slave. Standardising on asynchronous-assert, synchronous-release, active-low across all three would remove it.

The design philosophy held. Correctness was prioritised over feature count: the feature set is deliberately narrow, but everything inside it is verified with automated checks rather than visual inspection. The modular structure delivered what it promised — three blocks developed in parallel, each independently correct — and the cost of that parallelism was the interface drift noted above, a reminder that frozen interfaces are best enforced by a shared package rather than by agreement in a meeting.

Final statement

RTL simulation and FPGA synthesis were completed. Physical FPGA validation and bitstream programming were not performed because hardware was unavailable.

Appendix A — Simulation Transcript Summaries

The full transcripts are available in the repository. The closing summary of each run is reproduced below, since that is the part that determines pass or fail.

A.1 I²C master testbench

```
=====
          I2C MASTER VERIFICATION SUMMARY
=====
TOTAL TESTS : 14
PASSED      : 14
FAILED      : 0
=====
          ALL TESTS PASSED
=====
** Note: $finish : i2c_master_tb.sv(327)
Time: 1710250 ns   Instance: /i2c_master_tb
```

A.2 I²C slave testbench

```
PASS: WRITE: address ACK received
PASS: WRITE: data ACK received
PASS: WRITE: rx_data == 0xA5
PASS: READ:  address ACK received
PASS: READ:  data received == 0xA5
PASS: NACK handling: unexpected master ACK does not hang/corrupt bus
PASS: OPEN-DRAIN: bus reads LOW when driven low by either side
PASS: OPEN-DRAIN: bus releases HIGH (pull-up) when nobody drives low
=====
          I2C SLAVE VERIFICATION SUMMARY
=====
TOTAL TESTS : 22
PASSED      : 22
FAILED      : 0
=====
          ALL TESTS PASSED
=====
Time: 2304330 ns   Errors: 0, Warnings: 0
```

Appendix B — FSM Encodings

Master state	Enc	Slave state	Enc
IDLE	0	IDLE	0
START	1	RECV_ADDR	1
SEND_ADDR	2	ADDR_ACK	2
ADDR_ACK	3	WRITE_DATA	3
WRITE_DATA	4	DATA_ACK	4
DATA_ACK	5	READ_DATA	5
READ_DATA	6	READ_ACK	6
MASTER_ACK	7	WAIT_STOP	7
STOP	8	—	—
DONE	9	—	—
ERROR	10	—	—

Table 26 — State encodings, matching the numeric states printed in the transcripts.